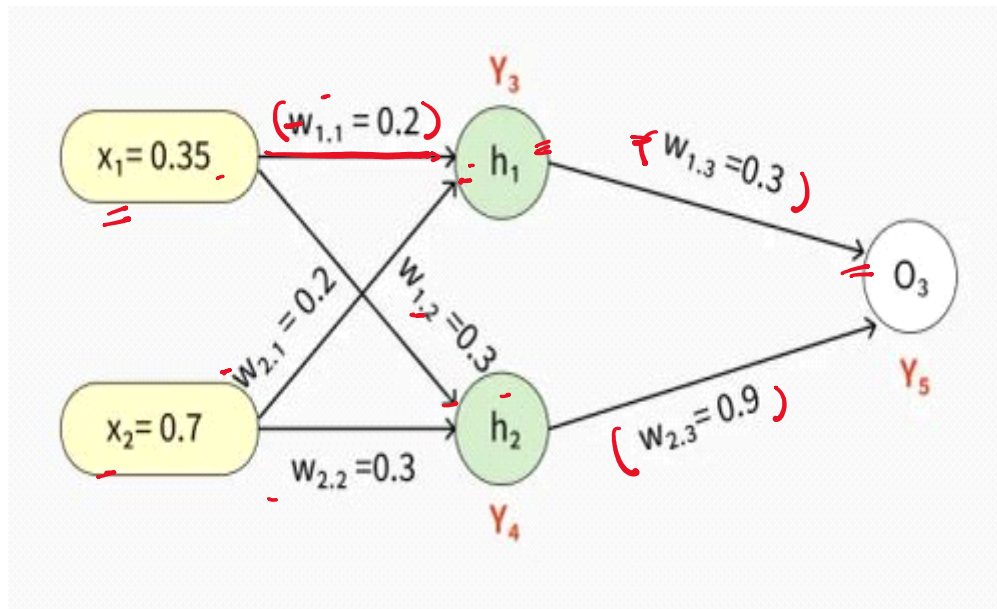


- 1. What is Quantization?**
- 2. Why do We Use Quantization?**
- 3. Difference between RAM and VRAM**
- 4. Data Types and their memory consumption**
- 3. What is Precision in Deep Learning?**
- 4. Formula of Quantization and Dequantization**
 - 4.1 Symmetric Quantization
 - 4.2 Asymmetric Quantization
 - 4.3 Per-Tensor Quantization
 - 4.4 Per-Channel Quantization
- 5. Type of Quantization**
 - 5.1 PTQ
 - 5.1.1 Static PTQ
 - 5.1.2 Dynamic PTQ
 - 5.2 QAT
 - 5.3 Calibration data
 - 5.4 Quantization Error
- 6. Practical Example with Neural Network**
- 8. Advance LLM Quantization Introduction**
- 9. GPTQ(Gradient Post-Training Quantization)**
- 10. AWQ (Activation-aware Weight Quantization)**
- 11. GGML(Georgi Gerganov's Machine Learning)**

- 12. GGUF(Gerganov's General Unified Format):
- 13. Practical Example with LLMs

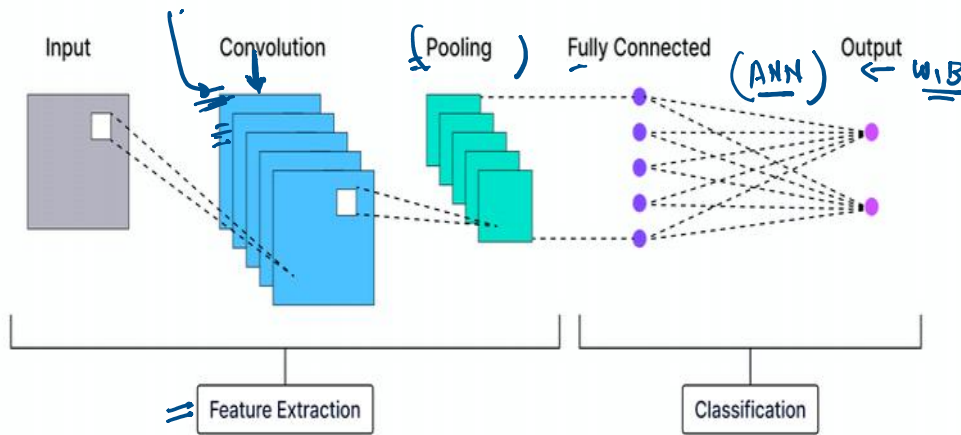
DIAGRAMS

Monday, August 4, 2025 6:26 PM



ANN
 $Q = W, B, Act$
 → higher precision ho. into lower precision
 → float
 (int)

The Architecture of Convolutional Neural Networks



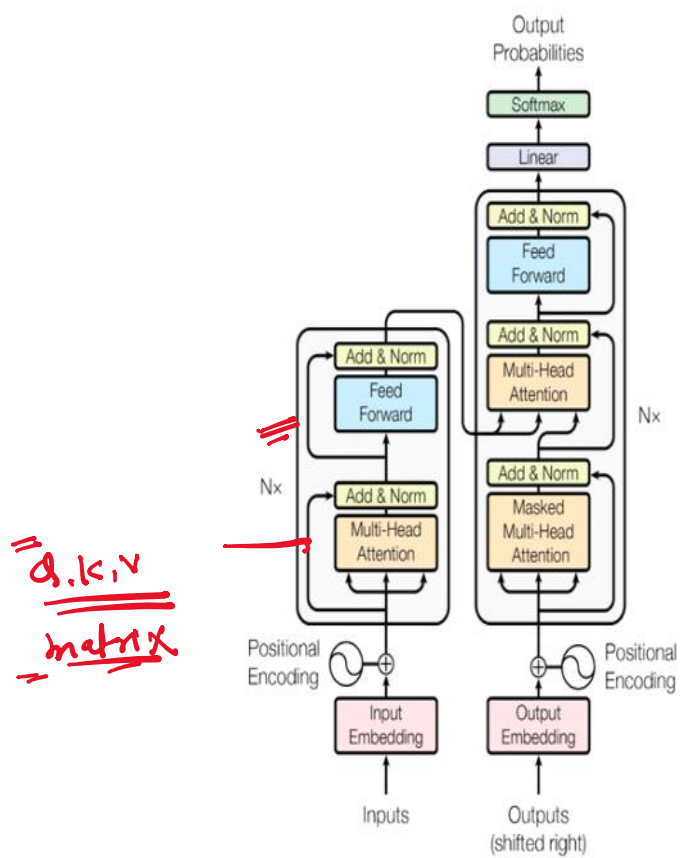


Figure 1: The Transformer - model architecture.

<https://github.com/Zhen-Dong/Awesome-Quantization-Papers>
<https://arxiv.org/html/2402.17764v1>

QUANTIZATION FULL COURSE

Monday, August 4, 2025 3:17 PM

1. What is Quantization?

Quantization is the process of converting a model weights and biases from high-precision numbers (like float32) to lower-precision numbers (like int8 or float16).
OR

Quantization is the art of shrinking models by converting high-precision numbers (weights and biases) into smaller ones, while trying to keep their intelligence intact.

What Do We Quantize in Model

Different model types have different types of quantizable weights:

Model Type	Quantizable Weights
ANN	Weights Between Hidden Layer
CNN	Convolution filters
RNN / LSTM	Recurrent weight matrices (e.g., hidden state transitions)
Transformer / LLM	Query, Key, Value matrices, Feed-Forward Network (FFN) weights

$$IIP \rightarrow A(IIP * w) \rightarrow OIP$$

Additional Notes:

- Biases can also be quantized (though optional)
- Activations are also quantized during inference (for speed) or training (in QAT)

Special Case: Attention Layers in LLMs

- In Transformers, attention layers are tricky to quantize
- Especially the softmax, which is sensitive to precision
- Care must be taken to avoid large accuracy drops when quantizing these parts

MODELS ARCHITECTURE

Simple Analogy:

Imagine you're storing prices of products:

In full detail: ₹1299.956 (float16)

After rounding: ₹1300 (int8 style)

$1300 - 1299.956 = 0.044$

You lose tiny details, but the core information remains. This is exactly what quantization does to model weights, biases and activations.

1. Mathematical Value / PI / Float Examples

Value	Precision Level
3.14159265	High precision (float32)
3.14	Low precision (float16)
3	Lowest precision (INT 8)

$$= 87.5793 - 87$$
$$0.5793$$

2. Currency Example (e.g. Dollar / ₹ Rupee)

Value	Precision Type
₹87.5793	High precision (float32-like)
₹87.573	Medium precision (float16-like)
₹87	Low precision (int8/int4-like)

3. Weight Measurement Example

Value	Precision Type
60.197 kg	High precision
60.2 kg	Medium precision
60 kg	Low precision

$$8:15$$
$$8:15:32$$
$$8:15:32.0000$$

=====First Part=====

2. Why do We Use Quantization?

Goal	How Quantization Helps
Shrink Model Size	Convert GB models into MB
Save RAM/VRAM	Load large models on small GPUs
Faster Inference	Smaller integers → faster matrix ops
Ease Deployment	Enables mobile/IoT inference/ Lower config Server/ Edge Device
Lower Serving Cost	Less VRAM use = cheaper GPU serving

Why Quantization Matters

Reduce Model Size

- FP32 (float) weights = 4 bytes per number
- INT8 weights = 1 byte per number
- This gives up to 4x smaller model size

Lower Memory (RAM/VRAM) Usage

- Quantized models consume significantly less memory
- Load large models on small GPUs (e.g., LLaMA 7B on 6GB GPU)
- Run more concurrent inferences on the same machine

- Quantized models consume significantly **less memory**
 - Load **large models on small GPUs** (e.g., LLaMA 7B on 6GB GPU)
 - Run **more concurrent inferences** on the same machine

Load Larger Models on Limited Hardware

- VRAM on your GPU is limited (e.g., 6GB, 12GB).
- Quantization allows loading much **larger models** in the same memory.
- Less RAM/VRAM usage = cheaper inference + faster loading.**

Example:

A 1GB model in FP32 → becomes **~250MB** in INT8.

Deploy on Edge Devices

- Phones, drones, IoT devices have **limited memory** and **no GPU**
- Quantized models enable **on-device AI**, such as:
 - Face detection on smartphones
 - Voice assistants like Alexa (offline mode)

Reduce Cloud Costs

- Smaller models → cheaper:
 - Storage**
 - Bandwidth**
 - Compute**
- Great for:
 - SaaS apps**
 - APIs**
 - Large-scale inference servers**

Accelerator Hardware Optimization

- Hardware like **TPUs, NVIDIA Tensor Cores, Intel AVX512** are optimized for **INT8/INT4**
- Quantization helps **fully utilize** these hardware accelerators

=====Second Part=====

What is RAM?

RAM is your **computer’s main memory**

It stores:

- OS processes
- Program data
- Neural network model (if running on CPU)

Example:

- You open Chrome + — these live in RAM
- Model running on CPU → RAM used
- Python Program Execution → RAM used

What is CPU (Central Processing Unit)?

Role: The “brain” of your computer — executes general-purpose instructions.

Example:

- Running Python scripts
- Browsing the web

What is VRAM?

VRAM is **dedicated memory on your GPU**

Stores:

- Model weights
- Activations during forward/backward pass
- Gradients (during training)
- Images / tensors (during inference)

Used by GPU (like NVIDIA, AMD)

Example:

- Model running on GPU (e.g. LLaMA) → weights loaded in **VRAM**
- If VRAM is low → model crashes (OOM)

What is GPU (Graphics Processing Unit)?

- Role:** A highly parallel processor optimized for graphics & matrix/tensor operations.
- Example:**
 - Running LLaMA or Stable Diffusion models
 - Playing a high-end video game
 - Video editing / 3D rendering

Feature	RAM	VRAM
Used by	CPU	GPU
Speed	Slower than VRAM	Extremely fast (optimized for tensors)
Application	Web, apps, CPU training	Deep learning inference/training on GPU
Capacity	8–64 GB typical	2–48 GB (depending on GPU)
Model hosted?	On CPU	On GPU
Quantization helps?	☒ Yes (smaller RAM needed)	☒ Yes (smaller VRAM needed)

Data Types with Memory Consumptions + Examples

- ◇ 8 bits = 1 byte
- ◇ float32 = 32 bits = 4 bytes per number

Data Type	Meaning	Size in Memory	Precision	Example Value	Precision Meaning	Usage in DL
float64	64-bit floating point (FP64) <i>(optional)</i>	8 bytes	Very High	3.141592653589793	~15–16 digits (scientific use)	Scientific computing, double precision
float32	32-bit floating point (FP32)	4 bytes	High	3.14159265	~7 decimal digits (e.g., 0.1234567)	Training & baseline
float16	16-bit floating point (FP16)	2 bytes	Medium	3.14	~3–4 decimal digits	Mixed precision training (NVIDIA AMP)
bfloat16	Brain float 16 (Google TPU)	2 bytes	Medium	3.14 (less mantissa)	~3–4 decimal digits	Training on TPUs
fp8	8-bit float (E4M3 or E5M2 variants)	1 byte	Low-Medium	~2.75 (approx)	~2–3 digits (approx)	NVIDIA Hopper GPUs, inference/training
nf4	Normal Float 4 (log-based float)	0.5 bytes	Very Low	0.5, 1.0, 2.0	Very coarse float values	bitsandbytes, GPTQ/AWQ
int8	8-bit integer	1 byte	Low	-128 to +127	No decimal (discrete values only)	Quantized inference
int4	4-bit integer	0.5 bytes	Very Low	-8 to +7	Very coarse discrete values	GPTQ, AWQ, extreme quantization
int32	32-bit integer	4 bytes	Exact	123456	Integer precision (no rounding)	Indexing, padding, tensor ops
bool	Boolean (True/False)	1 bit	None	True, False	Binary only (0 or 1)	Attention masks, binary outputs

Let's check some facts:

Let's say you have a neural net with **100 million parameters**(Weights and Biases)

Data Type Memory Required

float32	100M × 4B = 400 MB
float16	100M × 2B = 200 MB
int8	100M × 1B = 100 MB
int4	100M × 0.5B = 50 MB

Deep learning engineers **intentionally switch data types** based on use-case:

- **FP32** → Full training
- **FP16/BF16** → Mixed training
- **INT8/INT4** → Inference

=====Third Part=====

3. What is Precision in Deep Learning?

Precision means:

"How exact, detailed, and accurate a number can be stored"

"Precision means how finely a number is stored."

Precision Levels – Real-World Value Examples

"The more decimal values a number can store, the higher the precision."

"Float32 offers high precision but consumes more memory. INT8 offers lower precision, but it's much faster."

1. Mathematical Value / PI Examples

π

Value	Precision Level
3.14159265	High precision (<i>float32</i>)
3.14	Low precision (<i>float16 or int8</i>)

2. Currency Example (e.g. Dollar \$ / ₹ Rupee)

Value	Precision Type
₹87.5793	High precision (<i>float32-like</i>)
₹87.57	Medium precision (<i>float16-like</i>)
₹87	Low precision (<i>int8/int4-like</i>)

3. Weight Measurement Example

Value	Precision Type
60.197 kg	High precision
60.2 kg	Medium precision
60 kg	Low precision

Technical View:

- **float32** = 32 bits
 - Can store very **fine decimal values** (like 0.00000001)
- **int8** = 8 bits
 - Can store only **whole numbers** from -128 to 127
 - No decimal → Low precision

So,

- **float32** is **high precision**
- **int8** is **low precision**

Why Does Precision Matter?

- During **training**, we need **high precision** to learn small gradients and improve accuracy.
- During **inference**, we can use **low precision** (e.g. int8), because we only need fast predictions — even if they are **slightly less accurate**.

Format	Bit Width	Used For	Precision Level	Memory	Speed	Accuracy (Numerical)
FP32	32-bit	Full precision training	◇ High (~7 decimal digits)	High	Slow	✓ Best
FP16	16-bit	Mixed precision training	◇ Medium (~3–4 decimal digits)	Medium	Faster	▲ Slight drop
INT8	8-bit	Most common for quantization	○ Low (no decimals, -128 to +127)	Low	Fastest	▽ Small accuracy loss
INT4 / NF4	4-bit	Research, LLM compression (bnb/GPTQ)	▽ Very Low (coarse discrete/float values)	Very Low	Ultra-Fast	✗ Moderate accuracy loss
FP8	8-bit	NVIDIA Transformer inference (H100)	◇ Low-Medium (~2–3 decimal digits)	Low	Fast	▲ Slight drop

Additional info:

Precision ≠ Accuracy — What Does It Mean?

◇ Precision

"How finely or in how much detail a number is represented or stored."

- Related to **data format** (float32, int8, etc.)
- Example:
 - 60.123456 → **high precision**
 - 60 → **low precision**

◇ Accuracy

"How close is the prediction/output to the correct/true value."

- Related to **model performance**
- Example:
 - Actual label = 1, model predicts 0.97 → **high accuracy**
 - Predicts 0.4 → **low accuracy**

Real-Life Analogy:

Imagine you're throwing darts 🎯

- **Precision** = How tight your dart group is (clustered close together)
- **Accuracy** = How close they are to the **bullseye**

You can have:

- 🎯 High precision, low accuracy: all darts tightly grouped but far from target
- 🎯 Low precision, high accuracy: darts scattered but average hits the center

Best: high precision + high accuracy

Key Point:

A model can be low precision but high accuracy

Example:

- You quantize a model from float32 to int8 (precision ↓)
- But model still gives **correct output** (accuracy maintained)
- So, **low precision ≠ low accuracy**

=====Forth Part=====

4. Formula of Quantization and Dequantization

Quantization Formula (Uniform Linear Quantization)

Quantization (Float → Int)

$$x_q = \text{round}\left(\frac{x}{\text{scale}}\right) + \text{zero_point}$$

Where:

- x = original float value
- x_q = quantized integer (e.g., int8)
- **scale** = how much range one int step covers
- **zero_point** = which integer represents 0.0 in float
- **round()** = nearest integer

Dequantization (Int → Float)

$$x = (x_q - \text{zero_point}) \times \text{scale}$$

This is used during inference to convert quantized weights back to usable float values inside the neural network.

What is the Goal of Quantization?

To store **real numbers** (like float32: 1.23, -0.85, 2.77...) in **less memory** using small integers such as -128 to +127 or 0 to 255.

Math behind quantization:

- **Scale & Zero-point**
- **Clipping and rounding**
- **Range mapping**

What's the Problem?

Float values can sometimes fall into different ranges:

- Sometimes in range like [-2.0, +2.0]

- Sometimes only positive, like [0.5, 2.3]
- Sometimes unbalanced like [-5.7, 8.3]

To solve this — we use `scale` and `zero_point`:

$$x_q = \text{round}\left(\frac{x}{\text{scale}}\right) + \text{zero_point}$$

Symmetric vs Asymmetric Quantization Explained

♦ Symmetric Quantization:

Definition:

- `zero_point` = 0
- You center the data around 0
- You assume the data is in range like [-X, +X]
- **Quantization range:** [-128, +127] (for INT8)

Tensor Range Quant Range Quant Type Zero Point

[-2.0, +2.0]	[-128, 127]	Symmetric	0
[0.5, 2.5]	[0, 255]	Asymmetric	Non-zero
[-3.5, 8.2]	[0, 255]	Asymmetric	Non-zero

Example:

Let's say your model weights lie between [-2.0, +2.0]

→ Equal spread of negative and positive values

→ Center = 0

☒ So here, **symmetric quantization** is the best choice

$$x_q = \text{round}\left(\frac{x}{\text{scale}}\right)$$

`zero_point` = 0 (no shifting needed)

Example For Symmetric :

Let's say:

- Float value $x=1.23$
- Quant range = INT8 = [-128, +127]
- Min/Max of tensor = [-2.0, +2.0]

Step 1: Calculate scale

$$\text{scale} = \frac{\max - \min}{q_{\max} - q_{\min}} = \frac{2.0 - (-2.0)}{127 - (-128)} = \frac{4.0}{255} \approx 0.0157$$

Step 2: No zero point

Since symmetric quantization Hai:

`zero_point`=0

Step 3: Quantize

$$x_q = \text{round}\left(\frac{x}{\text{scale}}\right) = \text{round}\left(\frac{1.23}{0.01575}\right) \approx \text{round}(78.1) = 78$$

☒ 78 is the **quantized integer representation** of float 1.23

Step 4: Dequantize

$$x = x_q \times \text{scale} = 78 \times 0.01575 \approx 1.229$$

You get back approximate float: ~**1.229**, very close to 1.23

♦ Asymmetric Quantization:

Definition:

- `zero_point` ≠ 0 (non-zero shift)
- Used when data is **not centered around 0** (e.g., only positive values like [0.5, 2.5])
- **Quantization range:** [0, 255] (e.g., uint8)

$$x_q = \text{round}\left(\frac{x}{\text{scale}}\right) + \text{zero_point}$$

Example:

If model weights are only positive: [0.5, 2.5]

Center = 1.5 (not 0)

☒ So in this case, **asymmetric quantization** is preferred

`zero_point` ≠ 0 — this adjusts the mapping so the minimum float value aligns near 0 in the quantized space

Example For Asymmetric :

Parameter	Value
Float value x	1.23
Quant range	[0, 255] → using uint8
Min/Max of tensor	[0.5, 2.5]

Step 1: Calculate scale

$$\text{scale} = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}} = \frac{2.5 - 0.5}{255 - 0} = \frac{2.0}{255} \approx 0.00784$$

Step 2: Calculate zero_point

$$\begin{aligned}\text{zero_point} &= \text{round}\left(-\frac{r_{\min}}{\text{scale}}\right) \\ \text{zero_point} &= \text{round}\left(-\frac{0.5}{0.007843}\right) = \text{round}(-63.75) = \boxed{-64}\end{aligned}$$

Since zero_point cannot be negative for uint8, we clip it:

$$\text{zero_point} = \max(0, \min(255, -64)) = 0$$

Step 3: Quantize x = 1.23

$$x_q = \text{round}\left(\frac{x}{\text{scale}}\right) + \text{zero_point} = \text{round}\left(\frac{1.23}{0.00784}\right) + 0 = \text{round}(156.887) \approx 157$$

Step 4: Dequantize

$$x = (x_q - \text{zero_point}) \times \text{scale} = (157 - 0) \times 0.00784 \approx 1.23$$

Parameter	Value
Float value x	1.23
Quant range	[-128, 127] → INT8
Tensor Min/Max	[0.5, 2.5]

◆ Step 1: Compute Scale

$$\text{scale} = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}}$$

Substitute:

$$\text{scale} = \frac{2.5 - 0.5}{127 - (-128)} = \frac{2.0}{255} = \boxed{0.007843}$$

◆ Step 2: Compute Zero Point

$$\begin{aligned}\text{zero_point} &= \text{round}\left(-\frac{r_{\min}}{\text{scale}}\right) \\ \text{zero_point} &= \text{round}\left(-\frac{0.5}{0.007843}\right) = \text{round}(-63.75) = \boxed{-64}\end{aligned}$$

◆ Step 3: Quantize float value x = 1.23

$$\begin{aligned}x_q &= \text{round}\left(\frac{x}{\text{scale}}\right) + \text{zero_point} \\ x_q &= \text{round}\left(\frac{1.23}{0.007843}\right) + (-64) = \text{round}(156.8) - 64 = 157 - 64 = \boxed{93}\end{aligned}$$

Most quantized models use INT8 or INT4, not UINT8. And we rarely check tensor distribution (centered or not)

Hardware Preference — INT8/INT4 is Standard

- Almost all modern accelerators (CPU, GPU, NPU, TPU) are optimized for **signed int8/int4**:
 - **AVX2 / AVX-512 (Intel)** → supports int8
 - **NVIDIA TensorRT / Ampere / Hopper** → optimized for int8, int4
 - **ARM / Qualcomm chips (mobile)** → expect int8
 - **LLMs (GPTQ, AWQ)** → heavily use int4 / int8
 - **uint8** lacks widespread hardware support.

Framework Defaults

- Libraries like torch.quantization, onnxruntime, transformers, etc. default to **symmetric int8**.
- Even when using asymmetric quantization, they **still keep data in int8** but shift it with non-zero zero_point.
 ⚠️ So even in "asymmetric quantization", **they use int8, not uint8** most of the time.

Simplicity and Memory Alignment

- int8 has a **symmetric range** (-128 to +127) which makes it easy to:
 - Zero-center activations
 - Align quantized data in memory
- Frameworks like GPTQ, bitsandbytes use **quantized linear algebra** that works with int8/int4, not uint8.

Real-world model weights are mostly symmetric

- Most transformer weights like:
 - Query / Key / Value
 - Feedforward layers
 - Conv filters
 Are centered around 0
- So **symmetric quantization using int8 works very well**
 Only some embeddings / biases / activations may be asymmetric — handled with **per-channel quant + offset**

Term	Meaning
Quantization Error	Difference between real and quantized-dequantized value
Calibration	Forward-passing real data to compute min/max ranges for quantization

Per-Tensor Quantization

- ☒ **Definition:**
 - The **entire tensor** (e.g., full weight matrix) shares a **single scale and zero-point**.
- Use Case:**
 - Simpler and faster.
 - Often used for **activations**.
 - But less accurate, especially if different channels have different value ranges.

Example:
 Imagine a weight tensor of shape (4, 3):

```
[
  [0.1, 0.2, 0.3],
  [1.5, 2.0, 1.8],
  [0.05, 0.07, 0.08],
  [3.0, 2.5, 3.2]
]
```

In **per-tensor**, we calculate **1 scale + 1 zero-point** for all values above.
 Even though the ranges vary across rows, they all share the same quantization formula.

2. Per-Channel Quantization

- ☒ **Definition:**
 - Each **channel** (e.g., each row in weight matrix) gets its **own scale and zero-point**.
- Use Case:**
 - Used **mostly for weights** in CNNs, LLMs, etc.
 - More accurate.
 - Especially useful when each channel has a different range of values.

Example:
 Using same matrix:

```
[
  [0.1, 0.2, 0.3],    # Channel 1 → small range → scale1
  [1.5, 2.0, 1.8],    # Channel 2 → medium range → scale2
  [0.05, 0.07, 0.08], # Channel 3 → tiny range → scale3
  [3.0, 2.5, 3.2]     # Channel 4 → large range → scale4
]
```

In **per-channel**, we compute **4 different scales** and **4 zero-points** (one for each row).
 This allows fine-tuned quantization and **reduces loss in accuracy**.

Feature	Per-Tensor	Per-Channel
Granularity	Single scale/zero-point	One scale/zero-point per channel
Speed	Faster (simpler)	Slightly slower
Accuracy	Lower	Higher
Use Case	Activations	Weights (especially in LLMs, CNNs)
Preferred for	Small models or quick inference	High accuracy + transformer models

- ☒ **In Practice (LLMs & CNNs):**
 - LLMs** mostly use:
 - Symmetric + Per-Channel Quantization for weights (efficient & accurate).
 - CNNs/activations** might use:
 - Asymmetric + Per-Tensor Quantization for dynamic range handling.

In Transformers (like LLaMA, BERT):

- Query, Key, Value**, and **FFN weights** are **quantized per-channel** (row-wise) to preserve model behavior.
- This is usually **symmetric + per-channel**.

- Summary**
- Per-tensor** = **simple, fast, but may lose accuracy** if value distribution varies.
 - Per-channel** = **precise, handles varying ranges well** — best for large models and weights.

Bonus:

What is Quantization Error?

Quantization error means:

The small difference between original float value and quantized (int) value after you quantize and then dequantize it.

Why it happens?

Because you're trying to represent a high-precision value (e.g., 3.14159265) using a **low-precision integer** (e.g., int8 only has 256 possible values).

Example:

Suppose you quantize a float:

- Original: $x = 1.23$
- After quantization + dequantization: $x' = 1.21$

Then:

$$\text{Quantization Error} = x - x' = 1.23 - 1.21 = 0.02$$

Implication:

- Too much error $\rightarrow$ model accuracy drops
- That's why quantization must be done **carefully** (especially for LLMs and sensitive models)

What is Calibration?

Calibration is:

The process of analyzing real data to find accurate min/max values of tensors for quantization.

It's used in **Post Training Quantization (PTQ)**.

How?

- During calibration, you **pass a few batches of real input data** (no training, just forward pass)
- This helps the quantizer figure out:
 - Tensor ranges (min/max)
 - Which layers can use symmetric or asymmetric quant
 - Best scale & zero-point

Why calibration is important:

- Without calibration, wrong min/max values may cause **overflow or clipping**
- Model output becomes garbage!

```
model.eval()
model.qconfig = torch.quantization.get_default_qconfig('fbgemm')
torch.quantization.prepare(model, inplace=True)
# 🔄 Calibration step:
for batch in calibration_data:
    model(batch) # Just forward pass

torch.quantization.convert(model, inplace=True)
```

- **Zero-point is always 0**
This means the quantized value 0 corresponds to the real-world value 0.
- **Range is centered around 0**
For example, in int8, the range is typically [-127, 127].
- **Used when data is centered around zero** (like weight tensors).
- **Simpler math**, better for hardware acceleration (e.g., matrix multiplies).
- **Preferred in LLMs**, especially with **per-channel quantization**.
- **Zero-point is NOT zero**
Used when the data distribution is not centered around 0 (e.g., activation outputs).
- **Range is from [min, max]**
e.g., [0, 255] for uint8 or mapped real-world values.
- More flexible for **non-centered distributions**, like ReLU activations.
- Adds extra steps in inference math (subtracting zero-point), but works better for **diverse value ranges**.

=====Fifth Part=====

5. Type of Quantization

PTQ Post-Training Quantization

- SQ STATIC Quantization
- DQ Dynamic Quantization

QAT Quantization-Aware Training

PTQ = Post-Training Quantization

The model has already been trained, and now we are quantizing it (without retraining).

QAT = Quantization-Aware Training

"The model is trained with quantization in mind — it learns to handle quantization during training itself."

Intuition First: Easy Analogy

Imagine you're training a cricket player to bat:

🔪 PTQ: "Train with perfect bat, play with cheap bat later"

- You train with a high-quality bat (float32 weights)
- But in the match, you suddenly give him a plastic bat (int8)
- He might still perform okay, but maybe not perfect — he never *practiced* with this bat!

🔪 QAT: "Train and play with the same cheap bat"

- You train the player from day one with the exact bat (int8) he'll use in matches
- So he *learns how to handle its limitations*
- Outcome: more consistent performance

What is Calibration Data? (Simple Words)

It's just **some sample input data** that you run through the model to **measure the activation range (min/max)**.

Example:

Calibration Data = Just **dummy inputs** used to find out what activation range the model typically produces.

Let's say your model's hidden layer gives values between **(−3.5 to +5.7)**

Now You'll use this range to map it into **INT8 format (−128 to +127)**.

Feature	PTQ	QAT
When it's applied	After model training is done	During training (starts with float model)
What it does	Quantizes weights and activations after training	Simulates quantization during training
Model learning aware?	No — model never saw int8 during training	Yes — model learns to adapt to int8 errors
Accuracy impact	Slight accuracy drop (1–5%)	Minimal or no accuracy drop
Use-case	For fast compression after training	For production-grade quantization (e.g. edge)
Training time	No retraining needed — very fast	Retraining required — slower
Toolkits	torch.quantization.prepare/convert, GPTQ, BnB	prepare_qat, AutoQAT, TensorRT-QAT
Complexity	Easy to implement	More complex (fake quant, training config)

Model	Trained On	Predicts like
Float32	Full precision	$3.56 + 1.82 = 5.38$
PTQ	Float precision, then INT8 inference	$3 + 2 = 5$ ☒ (approx. ok)
QAT	Trained with INT8 simulation	$3 + 2 = 5$ ☒ (learns best rounding)

Aspect	PTQ	QAT
Training needed	No	Yes
Accuracy loss	Some (1–5%)	Minimal or none
Speed	Very fast	Slower (retraining needed)
Ease of use	Easy	Moderate to hard
Activation quant?	Optional or estimated	Accurate (learned)
Real int8 backend?	Yes after convert()	Yes after convert()
Toolkits	torch.quantization, GPTQ, bnb	prepare_qat, convert in PyTorch

Scenario	Best Option
Compress model for faster inference	PTQ
Deploy on edge/mobile (high accuracy needed)	QAT
Need quick model size reduction	PTQ
You're training from scratch or fine-tuning	QAT
LLM inference only	PTQ (GPTQ, AWQ)

What happens inside a model:

- **Weights & Biases** → These are learnable parameters of the model.
- **Activation values** → These are the outputs from each hidden layer after applying the activation function.
- **Forward Pass** → The input passes through the layers to produce the output. (This is what happens during inference.)
- **Backward Pass** → Gradients are calculated and used to update the weights. (This happens only during training.)

Type	Quantized Parameters	Calibration	Common Name
Static PTQ	Weights ☒ + Activations ☒	☒ Yes	Static Quantization
Dynamic PTQ	Weights ☒ only	☒ No	Dynamic Quantization

Normal or Partial PTQ = Weight only Quant

Static PTQ = Calibration = Activation + Weight Quant

Dynamic PTQ = No Calibration = Weights are pre-quantized (saved as INT8), but activations are quantized temporarily during inference time (on-the-fly), not permanently — no calibration required."

QAT = Training time pe quantization ka practice

Type	Quantize Weights	Quantize Bias	Quantize Activations	Needs Calibration	Training Involved
PTQ (Static)	☒	☒	☒	☒ Yes	☒ No
PTQ (Dynamic)	☒	☒	☒ (done at runtime)	☒ No	☒ No
QAT	☒	☒	☒	☒ No	☒ Yes (full training)

Feature	Dynamic Quantization	Static Quantization
Weights	INT8 (pre-quantized)	INT8 (pre-quantized)
Activations	Quantized on-the-fly	Quantized beforehand using calibration
Calibration	☒ Not needed	☒ Needed
Accuracy	Moderate	Better
Speed	Fast	Faster (usually)

PTQ (Post-Training Quantization)

The model is already trained (in FP32), and then you apply quantization afterward.

◊ **Types:**

- **Weights Quantization** — ☒ Done
- **Bias Quantization** — ☒ Done (usually)
- **Activation Quantization** — ☐ It depends:
 - If you use **Static Quantization (SQ)** → ☒ Activations are quantized
 - If you use **Dynamic Quantization (DQ)** → ☒ Activations are *not* pre-quantized (they're quantized on-the-fly during inference)

STATIC Quantization (SQ)

☒ A type of PTQ where **both weights and activations** are quantized.

A **calibration dataset** is used to calculate the min/max values of activations.

Process:

- You perform a **forward pass** of the model on a small calibration dataset.
- For each layer, you **collect the min/max range of activation values**.
- Then, you use that to **pre-quantize activations** into **INT8 format**.

Static Quantization = requires calibration + quantizes both weights and activations.

DYNAMIC Quantization (DQ)

☒ Also a type of PTQ, but **lighter** and faster to apply.

- Only **weights** are quantized (to **INT8**)
- **Activations remain in FP32** and are quantized **on-the-fly** during inference

Dynamic Quantization =

No calibration required,

Only weights are quantized,

Activations are dynamically quantized during inference.

QAT (Quantization Aware Training)

During training, the model is made to "*feel*" that it will be quantized later.

Inside QAT:

Forward Pass:

- Quantization is **simulated** (rounded off to INT8) during the forward pass
- → The model **learns to handle quantization error**

Backward Pass:

- **Backpropagation** happens on actual **FP32 weights**,
- but the **gradient is computed w.r.t. quantized values**
- This technique is called the "**Straight-Through Estimator (STE)**"

QAT =

Training + Simulate quantization during forward pass +

Backward pass on FP32, but loss comes from quantized output.

1 STATIC Quantization

Summary:

Weights and activations — both are quantized **ahead of inference** using a **calibration dataset**.

How it works:

- Model is trained in float32
- You provide a **few batches of real data** (calibration)
- It computes min/max of **weights & activations**
- Then it **quantizes both** statically

Used When:

- You need full optimization
- Target device supports full INT8 pipeline
- You can afford calibration step

Real-Life Analogy:

You test every camera setting before a shoot, lock all values (brightness, exposure) — ready to shoot.

Pros:

- Very optimized (speed + memory)
- Works well on mobile (qnnpack) & edge (fbgemm)

Cons:

- Needs calibration data
- Slight risk of poor accuracy if calibration is bad

2 DYNAMIC Quantization

Summary:

Weights are quantized **ahead of time**, but **activations are quantized on-the-fly** at runtime.

How it works:

- Weights are stored as int8
- Activations are converted to int8 **during inference**
- No calibration needed

Used When:

- You want **quick quantization** with minimal effort
- Deployment is CPU-only (e.g., AWS CPU, Raspberry Pi)

Real-Life Analogy:

You keep your camera in auto-mode. It adjusts lighting **dynamically** at each moment.

Pros:

- No calibration needed
- Easy to apply to pre-trained models (especially LLMs)

Cons:

- Slightly less optimized than static
- Works best for **fully connected layers**, not CNNs

3 QUANTIZATION-AWARE TRAINING (QAT)

Summary:

Simulates quantization **during training** so model learns how to handle low-precision.

How it works:

- Insert **fake quant/dequant ops** during training
- Backprop still uses float32, but forward uses simulated int8
- After training, you convert to real int8 model

Used When:

- Accuracy is super critical
- Deploying on tight-resource devices (edge, phones)

Real-Life Analogy:

You train a cricketer using a plastic bat from Day 1 — so he masters it, even in real matches.

Pros:

- Best accuracy
- Learns to handle quantization errors

Cons:

- Slower (requires retraining)
- More complex to set up

=====Sixth Part=====

6. LLM Advance Quantization Introduction

Term	Meaning & Clarification
FP	Floating Point
FP8 formats	E4M3 and E5M2 are two common FP8 formats:• E4M3: 4-bit exponent, 3-bit mantissa (used in NVIDIA Hopper)• E5M2: 5-bit exponent, 2-bit mantissa
INT	Integer (e.g., INT8, INT4 used for quantization)
NP	Normal Precision Float (not a standard term — clarify this if you're using it for your course)
PTQ	Post-Training Quantization — quantize a model after full training without re-training
QAT	Quantization-Aware Training — simulate quantization during training to preserve accuracy
GPTQ	Originally stands for " Gradient Post-Training Quantization ". It is a specific PTQ technique designed for transformers. People confuse it with "Generative Pretrained Transformer Quantization", but that is incorrect.
AWQ	Activation-aware Weight Quantization — focuses on scaling weights w.r.t activations for better accuracy in INT4/INT8
GGML	Georgi Gerganov's Machine Learning library for inference — supports CPU/GPU/backends
GGUF	Gerganov's General Unified Format — successor to GGML binary formats. Used to store quantized models (e.g., LLaMA, Mistral) for use in tools like llama.cpp. ! Some people say it stands for "GPT-Generated Unified Format" but that's a backronym , not official.

Quantization Workflow Sequence (Refined English + Technical)

1. **You perform quantization** — using any of the following methods:
 - **PTQ** (Post Training Quantization)
 - **QAT** (Quantization Aware Training)
 - **GPTQ** (Gradient-based PTQ for Transformers)
 - **AWQ** (Activation-aware Weight Quantization)
2. **Then you save the quantized model in a specific format**, such as:
 - **GGUF** (popular for the LLaMA + llama.cpp ecosystem)
 - **safetensors** (secure tensor format used in Hugging Face Transformers)
 - **ONNX** (used for inference with ONNX Runtime)
 - **TensorRT engine** (for NVIDIA-optimized inference)
3. **And finally, you need an engine to run the saved format**, such as:
 - **llama.cpp** (for GGUF models)
 - **GGML** (older format, now replaced by GGUF)
 - **vLLM** (used with safetensors for high-throughput inference)
 - **TensorRT-LLM** (optimized for INT4/INT8 inference on NVIDIA GPUs)

One-Line Summary (Crisp & Final)

- **PTQ, QAT** = *Quantization techniques* (how quantization is done).
- **GPTQ, AWQ** = *Quantization algorithms/libraries* (enhanced PTQ methods for LLMs).
- **GGUF** = *Model file format* (used for saving/exporting quantized models).
- **GGML** = *Inference runtime* (engine that runs quantized models like GGUF).

PTQ → GPTQ → GGUF → GGML

Refined Example Flow (End-to-End):

1. You have a **LLaMA-3 8B model** with **FP16 weights**.
2. You perform **quantization using GPTQ** (which is a type of **Post-Training Quantization**).
3. You **export the quantized model in GGUF format** (which is compatible with llama.cpp).
4. Finally, you **run the model using llama.cpp or a GGML-based engine** — on **CPU or Mac**, no GPU required.

Extra Notes (for depth):

- **GPTQ** = *Post-Training Quantization specifically for Transformers*
- **GGUF** = *Efficient model file format used by llama.cpp ecosystem*
- **GGML** = *Inference engine that consumes GGUF or older GGML format for CPU/low-resource inference*

Tool / Format	Short Refined Explanation
GPTQ	Post-Training Quantization method (weight-only) for LLMs. Hugging Face + AutoGPTQ integration allows fast 4-bit inference on GPUs. Good balance of speed & accuracy.
AWQ	Activation-aware Weight Quantization . Focuses on minimizing accuracy drop in 4-bit models by preserving important activation patterns. Often slightly better quality than GPTQ in same bit-width.

bitsandbytes	Popular Hugging Face-compatible library for 8-bit & 4-bit loading (LLM.int8(), nf4). Very easy for quick memory savings without retraining.
GGUF	Quantized model file format used in llama.cpp ecosystem. Stores weights efficiently in various bit-widths for fast CPU/Mac/low-VRAM GPU inference. Successor to GGML's .bin format.
GGML	C-based inference runtime for quantized models (e.g., LLaMA). Optimized for CPU and lightweight GPUs. Backbone for llama.cpp.
llama.cpp	High-performance C++ wrapper for GGML/GGUF models. Offers easy CLI tools + C/C++/Python APIs for running LLMs locally without heavy dependencies.
TensorRT-LLM	NVIDIA library for optimized GPU inference with quantized or FP16/BF16 models. Great for production servers.
vLLM	High-throughput LLM serving engine. Optimized for batching, paged attention, and multi-GPU scalability.
ONNX Runtime	Cross-platform runtime for models exported in ONNX format . Supports CPU, GPU, TensorRT execution with quantization.

☒ **Refined Summary (with clarity and categories)**

- Quantization Libraries:**
- **GPTQ (AutoGPTQ)** – Fastest for 4-bit quantization (Hugging Face compatible).
 - **AWQ** – 4-bit quantization with activation-aware optimization (better accuracy).
 - **bitsandbytes** – Enables INT8 / 4-bit loading directly in Hugging Face models.
- Model File Format:**
- **GGUF** – Efficient file format for storing quantized models (used with llama.cpp).
- Inference Runtimes:**
- **GGML** – C-based lightweight inference engine (runs quantized models on CPU/Mac).
 - **llama.cpp** – Wrapper around GGML with CLI + API interface for easy use.
- GPU/Server Deployment:**
- **TensorRT-LLM, vLLM, ONNX Runtime** – High-throughput GPU inference on server setups.

Can We Do PTQ/QAT on LLMs?
 Yes — **Post-Training Quantization (PTQ)** and **Quantization-Aware Training (QAT)** *are* both applicable to LLMs, but the process is significantly different from small-scale models like CNNs or MLPs.

PTQ on LLMs

In MLPs (simple case):
 $scale = (rmax - rmin) / (qmax - qmin)$
 $zero_point = round(-rmin / scale + qmin)$
 Done on small tensors (e.g., linear layers)
 Can use torch.quantization directly

- In LLMs:**
- Same core formula, but with **huge tensors**.
 - Quantization is done:
 - **Layer-wise** or **per-channel**
 - On weights like **Q/K/V, Feed Forward (FFN)**, etc.
 - Requires a **small calibration dataset** (~50–200 samples).
 - Very sensitive to precision loss

- Tools for PTQ on LLMs:**
- GPTQ (Gradient-aware PTQ)
 - AWQ (Activation-aware Weight Quantization)
 - AutoGPTQ (wrapper around GPTQ + Hugging Face)
 - bitsandbytes (8-bit & 4-bit loaders)

- QAT on LLMs**
In MLPs (simple case):
- Use QuantStub/DeQuantStub to simulate quant ops.
 - **Fake quantization** during training.
 - Backprop through **quantization ops** with minimal overhead.

- In LLMs:**
- Much more **complex and expensive**:
 - You simulate quant ops using **STE (Straight-Through Estimator)**.
 - Requires **full or partial retraining** of the model.
 - Requires **multi-GPU** or **TPU setups** (very compute-heavy).

- Tools for QAT on LLMs:**
- torchao (PyTorch QAT backend)
 - Intel Neural Compressor
 - NVIDIA TensorRT-LLM (QAT support)
 - OpenVINO (QAT & PTQ optimizations)
 - HQQ (High-Quality Quantization + QAT)
 - SmoothQuant (can be combined with QAT)

Component	Notes
◊ Activation quantization	Challenging due to softmax, layernorm, attention
◊ Weight quantization	Done on Q/K/V, FFN, etc., often with per-channel granularity
◊ Techniques	Dynamic, static, and fused quantization techniques are often combined

Aspect	MLPs	LLMs (e.g., GPT, LLaMA)
PTQ	Easy (torch.quantization)	Layer-wise, needs calibration, uses GPTQ/AWQ
QAT	Light training needed	Heavy retraining with STE, uses advanced toolkits

Activation Handling	Simple	Complex (softmax, attention layers)
Tools	Built-in torch	GPTQ, AWQ, bitsandbytes, torchao, TensorRT-LLM
Hardware Need	Single GPU	Multi-GPU/TPU for QAT

7. What is GPTQ?

GPTQ (Gradient Post-Training Quantization) is an **advanced Post-Training Quantization (PTQ)** method specifically designed for **Large Language Models (LLMs)** like **LLaMA, GPT, Falcon**, etc. It quantizes model **weights to low precision (e.g., INT4)** after training, **without any fine-tuning**, while still preserving the original model's accuracy as much as possible.

Key Characteristics

- **Advanced PTQ method** using gradient-based optimization
- Designed **especially for Transformers and LLMs**
- Targets **INT4/INT8** quantization for model compression and faster inference
- Does **not require retraining** the model

What Does “GPT” Mean in GPTQ?

Despite "GPT" in the name, **GPTQ actually stands for:**

Gradient Post-Training Quantization

Which means:

Use **gradients (second-order information)** during the quantization process to **minimize quantization error** — **without retraining** the model.

Core Idea Behind GPTQ

GPTQ treats quantization as a **matrix approximation problem**:

You're replacing the original full-precision weight matrix **W** with a quantized version **$\hat{W}$ (W-hat)**.

Objective:

Minimize the output difference caused by using **$\hat{W}$ instead of W**, i.e.

$$W \cdot X \approx \hat{W} \cdot X$$

Where:

- W = Original weight matrix
- $\hat{W}$ = Quantized weight matrix
- X = Input activations

Complete Roadmap for GPTQ

Module Topic

- 1 **What is GPTQ?**→ Theory, Intuition, and Real-World Use Cases
- 2 **Setup & Installation**→ Installing AutoGPTQ, Transformers, bitsandbytes, and dependencies
- 3 **Quick Demo**→ Perform 4-bit quantization on BERT / LLaMA using GPTQ (AutoGPTQ)
- 4 **Internals of GPTQ**→ How GPTQ computes optimal quantized weights (gradient-based second-order PTQ)
- 5 **Evaluation & Benchmarking**→ Analyze trade-offs: Accuracy vs Inference Speed vs Model Size
- 6 **Deployment & Inference**→ Export GGUF + Run quantized model via llama.cpp on CPU/Mac
- 7 **Troubleshooting & Best Practices**→ Calibration tips, common errors, hardware-specific notes

Concept	GPTQ Behavior
Type of Quantization	Post-Training Quantization (PTQ) — no model retraining required
Quantized Precision	Typically INT4 , occasionally INT3 or INT2 for extreme compression
Target Components	Linear layers within Transformer blocks — including Q/K/V , FFN , etc.
Core Idea	Quantize weights layer-by-layer using second-order optimization to minimize output error , not just weight error
Accuracy	High — performance remains very close to the original FP16 model
Speed	Much faster than QAT — ideal for post-training model deployment
Calibration Data	Required: around 50–200 representative text samples for accurate quantization

Intuition Behind GPTQ (Gradient Post-Training Quantization)

Typical PTQ:

- Post-Training Quantization (PTQ) usually just replaces full-precision weights with low-precision (e.g., INT4 or INT8) equivalents.
- It focuses on minimizing the **difference between original and quantized weights**.
- But this doesn't always ensure that the model's **final output** remains close to the original.

GPTQ is smarter:

1. **Tracks the error** that quantization introduces in each layer — **not just at the weight level**, but also how it affects the model's output.
2. **Uses second-order information** like gradients or **Hessian approximations** to understand the sensitivity of weights — i.e., which weights are more important for the model's performance.
3. GPTQ then selectively quantizes weights **in a way that minimizes the final output error**, not just weight-wise difference.

Quantized GPTQ models can be saved in various formats depending on your deployment pipeline:

Format	Description
.safetensors	Secure & efficient format used by Hugging Face ecosystem (e.g., AutoGPTQ)
.gguf	Converted format for use with llama.cpp (fast inference on CPU/Mac)
.pt or custom	PyTorch checkpoint format or internal formats for specific libraries

GPTQ Summary Table

Feature	GPTQ
Type	Post-Training Quantization (PTQ)
Target	Transformer-based LLMs (e.g., LLaMA, Falcon, GPT, BLOOM)
Precision	Typically INT4, also supports INT8, INT3
Speed	Very fast inference (esp. with ExLlama, llama.cpp, etc.)

Accuracy High (if properly calibrated and quantized)
Tools AutoGPTQ, ExLlama, Hugging Face Transformers, bitsandbytes

Library / Tool	Description
AutoGPTQ	Hugging Face compatible wrapper around GPTQ; works with Transformers easily.
GPTQ-for-LLaMA	Original low-level repo by Yao Fu et al. implementing GPTQ from scratch.
ExLlama & ExLlamaV2	Fast inference engines for GPTQ quantized models (especially on CUDA GPUs).
llama.cpp	Supports running GPTQ quantized GGUF models (CPU/Mac inference).

Step-by-Step Mathematics of GPTQ

1. Model Layer Representation

Let:

- $W \in \mathbb{R}^{n \times d}$ be the full-precision weight matrix.
- $X \in \mathbb{R}^{b \times n}$ be the calibration input (b = batch size).
- The output of the linear layer:

$$Y = XW^T$$

2. Quantization Goal

We want a quantized version $\hat{W}$ ($\hat{W}$) such that:

$$\text{Loss} = \|XW^T - X\hat{W}^T\|_F^2$$

Here:

- $\|\cdot\|_F^2$ is the **Frobenius norm** (sum of squared differences).
- The loss is minimized over **rows of W**, one at a time.

This is why GPTQ is **row-wise quantization**.

3. Hessian Matrix Approximation

To preserve the model's output better, GPTQ incorporates **second-order error correction** using the **Hessian matrix**:

- Approximate local curvature of the loss landscape w.r.t. weights.
- Hessian for each row i of W :

$$H_i = X^T X$$

This is a **covariance matrix** of the inputs. It tells us **which directions in W are more sensitive** to changes.

4. Blockwise Greedy Quantization

- Quantize weights **one at a time** in blocks.

- For each weight w_{ij} , choose quantized value $\hat{w}_{ij}$ that minimizes:

$$\text{Error Increase} \approx (w_{ij} - \hat{w}_{ij})^2 \cdot H_{jj}$$

If H is diagonalized, this is fast and local.

5. Error Compensation

After quantizing a weight w_{ij} , **update remaining weights** to compensate for the quantization error:

$$w_{ik} := w_{ik} - \frac{H_{jk}}{H_{jj}} \cdot (w_{ij} - \hat{w}_{ij})$$

This step is similar to **Low-Rank Approximation** where later rows compensate for errors introduced by earlier ones.

Step	Formula
Linear Output	$Y = XW^T$
Loss Function	$\ XW^T - X\hat{W}^T\ _F^2$
Hessian (Row-wise)	$H = X^T X$
Quant Error Estimate	$(w - \hat{w})^2 \cdot H$
Error Compensation	$w' = w - \text{correction}$

Calibration Dataset

- GPTQ requires a small set of **real inputs (50–200 samples)** to simulate real inference data.
- This helps compute X and hence H , making quantization more accurate.

GPTQ vs Standard PTQ: **Key Differences**

✅ Verified Comparison: Standard PTQ vs GPTQ (Advanced PTQ)		
Aspect	Standard PTQ (Basic)	GPTQ (Advanced PTQ)
Goal	Quantize weights to INT8/INT4	Same goal, but minimizes output error more precisely
Formula	$x_q = \text{round}\left(\frac{x}{\text{scale}}\right) + \text{zero_point}$ 🟡 Simple scalar rounding based on min/max	$\min_{\hat{W}} \ XW^T - X\hat{W}^T\ _F^2$ 🟡 Matrix approximation to reduce output deviation
Level	Mostly tensor-wise or channel-wise	Always row-wise quantization (per weight vector)
Error Awareness	❌ No modeling of how quantization affects final output	✅ Uses Hessian (2nd-order gradients) to estimate the error introduced
Error Compensation	❌ No	✅ Yes — adjusts quantization to compensate future layer errors
Calibration Data	Optional (often skipped)	✅ Mandatory (~50–200 text samples) to compute statistical structure like $X^T X$
Math Behind	Basic linear scaling	Linear algebra: matrix decomposition + 2nd-order correction
LLM Use Case	❌ Rarely sufficient for Transformer ⁿ LMs ↓	✅ Specifically designed for LLMs like LLaMA, GPT, Falcon, etc.

Simple Terms:

- **Standard PTQ** just compresses numbers layer-wise using **min/max scaling**.
- **GPTQ** is smarter: it checks **how much quantizing a weight will affect model outputs**, and avoids high-impact quantization errors using the Hessian and **greedy + compensatory logic**.

Real-Life Analogy:

- **Standard PTQ** = blindly shrink all clothes to medium size.
- **GPTQ** = check body shape, try fitting each shirt, and adjust the next one based on how the last one shrunk. Smart and adaptive!

8. AWQ: Activation-aware Weight Quantization

AWQ is a Post-Training Quantization (PTQ) method like GPTQ, but it improves output accuracy by also considering the **activation distribution** during quantization.

Feature	GPTQ	AWQ
Error Modeling	Weight-only error	Weight + Activation error
Uses Hessian	✅ Yes	❌ No (simpler & faster)
Calibration Needed	✅ Yes (50–200 samples)	✅ Yes (same)
Precision	INT4 (mainly)	INT4 (mainly)
Speed	Fast	Even faster (no second-order calc)
Accuracy	High	Very High (better than GPTQ in some LLMs)
Quant Target	Weights	Weights + Activation impact
Use Case	LLaMA, GPT	LLaMA, Mistral, ChatGLM, etc.

Intuition Behind AWQ

Instead of blindly quantizing weights, **AWQ checks how each weight affects the output, after going through activation functions** (like GELU, ReLU, etc). It then **prunes and rescales weights** so that they cause **minimum distortion** in output — especially on **important tokens** during inference.

Why is this Better Than GPTQ in Some Cases?

GPTQ	AWQ
Focus on pre-activation error	Focus on post-activation (real output) error
Uses second-order gradients (Hessian)	No gradients – lighter & faster
Slower on large models	Faster, no Hessian
Good for precision	Better for both precision & speed (especially in inference-only scenarios)

Real-Life Analogy

GPTQ = Measures how much *raw ingredients* changed

AWQ = Measures how the *final dish tastes* after cooking (activation)

✓ Summary of Mathematical Pipeline:

Step	Operation	Math
1	Activation Capture	$X \leftarrow \text{calibration data}$
2	Token Importance	Score high-impact tokens
3	Weight Scaling	$W' = \text{Scale}(W, \text{importance})$
4	Pruning (optional)	Remove low-importance weights
5	Quantization	$\hat{W} = \text{Quantize}(W')$
6	Final Output	$\hat{Y} = f(X\hat{W})$, minimize $\ f(XW) - f(X\hat{W})\ ^2$

AWQ Mathematics – Full Detailed Breakdown

Problem Setup:

In any Transformer-based LLM:

- X = Input activation matrix (tokens $\rightarrow$ hidden states)
- W = Weight matrix of a linear layer (Q, K, V, FFN etc.)
- f = Non-linear activation (GELU, ReLU etc.)
- $\hat{W}$ = Quantized weights

The output of a layer is:

$$Y = f(XW)$$

When quantized:

$$\hat{Y} = f(X\hat{W})$$

Goal of AWQ is to minimize output error:

$$\min_{\hat{W}} \|f(XW) - f(X\hat{W})\|_F^2$$

Ye GPTQ se different hai. GPTQ minimize karta hai:

$$\|XW - X\hat{W}\|_F^2 \quad (\text{before activation})$$

But AWQ includes non-linearity, so it's more realistic and better for preserving accuracy in LLMs.

Step-by-Step Explanation:

Step 1: Calibration

- 50–200 real text prompts ko model me input diya jaata hai.
- Ye prompts se **activation values** XXX collect kiye jaate hain.

Step 2: Token Importance Estimation

Some tokens contribute **more to final output** than others.

AWQ:

- Calculates per-token activation scores.
- Identifies "**important tokens**" – jinpe quantization ka impact kam se kam hona chahiye.

Step 3: Weight Rescaling & Pruning

Weight matrix WWW ko **token importance** ke hisaab se adjust kiya jaata hai.

Heuristic approach:

- Weights are **rescaled** and **non-important rows/columns pruned** before quantization.
- This is done **blockwise**, just like GPTQ, typically in **128-row blocks**.

$$\hat{W} = \text{Quantize}(\text{Prune}(\text{Scale}(W, \text{token importance})))$$

Step 4: INT4 Quantization

Once rescaled and pruned:

- INT4 quantization is applied.
- Same formula as standard PTQ:

$$w_q = \text{round}\left(\frac{w}{s}\right)$$

Where:

- s = scale factor for block
- w = original (rescaled) weight
- w_q = quantized weight (in INT4 range)

Why is this Better Than GPTQ in Some Cases?

GPTQ	AWQ
Focus on pre-activation error	Focus on post-activation (real output) error
Uses second-order gradients (Hessian)	No gradients – lighter & faster
Slower on large models	Faster, no Hessian

9. GGML(Georgi Gerganov's Machine Learning)

GGML: Georgi Gerganov's Machine Learning

- **GGML** originally stood for the library created by **Georgi Gerganov**, who is the creator of llama.cpp.
- It is **not** "GPT-Generated Model Language" (that's a common misinterpretation online).
- GGML is a **tensor library for machine learning on CPUs**, optimized for quantized inference.

Use case:

Used to run LLaMA, Whisper, GPT models on laptops/mobile devices using CPU. Initially supported .bin model files (quantized via GGML format).

<https://arxiv.org/abs/2210.17323>

1. What is GGML?

GGML stands for **Georgi Gerganov's Machine Learning** framework.

- ☒ It's a C-based tensor library optimized for **inference of quantized models** — especially for **CPU and Apple Silicon (Mac)**.
- ☒ GGML was *not* designed for training or GPU inference (though hacks exist).

2. Key Highlights

Feature	Description
 Language	Written in pure C (highly portable)
 Focus	Efficient <i>inference</i> of LLMs with quantized weights
 Quantization	INT8, INT4, custom quantizations like Q4_0, Q4_K, etc.
 Model Types	Works well with decoder-only LLMs like LLaMA, Falcon
 Hardware Optimized	CPU (x86, ARM), Apple Silicon (Metal backend)

3. What GGML Is Not Meant For

☒ Not meant for	Reason
GPU acceleration (by default)	No CUDA / ROCm support in core
Model training	No backward pass / autograd
Fine-tuning	Only forward/inference supported

4. Why GGML Became Popular

- ☒ It enabled **local inference of LLaMA models** on laptops.
- ☒ Used in tools like:
 - **llama.cpp** (main client)
 - **text-generation-webui**
 - **koboldcpp**

5. GGML vs GGUF

Feature	GGML Format	GGUF Format
☒ File type	.bin + external tokenizer files	.gguf (self-contained)
☒ Metadata	Minimal	Rich + embedded
☒ Tokenizer	Separate file needed	Embedded
☒ Quant formats	Limited	Wide support (Q2_K to Q8_0)
☒ Versioning	No formal versioning	GGUF has versioning

☒ **GGUF succeeded GGML** and is now the preferred format for llama.cpp.

6. GGML Format Structure

GGML stores models in .bin files with this format:

[Header]

- └─ Magic bytes: GGML
- └─ Version info (optional)

[Model Info]

- └─ Dimensions, number of layers, vocab size

[Tensor Blobs]

- └─ Each tensor has name, shape, data type (quantized), actual values
- Each weight is stored in its **quantized form**, e.g., INT4 or INT8.

7. Quantization Support in GGML

Quantization	Bit Width	Use Case
Q4_0	4-bit	Basic compression
Q4_K	4-bit	Smarter quant (like AWQ)
Q5_0	5-bit	Balanced speed & accuracy

Q8_0	8-bit	Near-FP16 quality
F16 / F32	16/32-bit	Full precision

Note: GGML does **not** use standard torch.quantization. It has its **own custom quant** schemes optimized for memory and cache locality.

8. Use with llama.cpp

Although GGML can be used alone, it’s mostly used **via llama.cpp**, which wraps its functionality for:

- Text generation
- Prompt processing
- Streaming outputs
- GPU offloading (partial)
- Quantized inference on CPU/Mac

Example command:

```
./main -m model.bin -p "Explain quantization in LLMs."
```

9. Tooling

Tool	Purpose
convert.py	Convert Hugging Face models to GGML
llama.cpp	CLI wrapper to run GGML models
text-generation-webui	Web interface using GGML backend

10. Transition to GGUF

- GGML format is **deprecated** for new projects.
- Convert your GGML .bin models to .gguf using scripts like:

```
python3 convert-llama-ggml-to-gguf.py
```

GGUF adds:

- Tokenizer support
- Quant format standardization
- Model info + metadata
- Compatibility with newer LLaMA variants

Summary

GGML	GGUF
Old quantized format	New unified format
.bin files	.gguf files
No tokenizer inside	Tokenizer embedded
No versioning	Versioned and standardized
Still works	Recommended to migrate ☒

10. GGUF(Gerganov's General Unified Format):

GGUF (GGML Unified Format) is a standardized file format designed for storing and deploying quantized large language models (LLMs), primarily used within the llama.cpp ecosystem. **“GGUF (a unified binary format from the GGML/llama.cpp ecosystem)** is a file format that streamlines the storage and deployment of quantized large language models (LLMs).

GGUF: GPT-Generated Unified Format

- GGUF stands for **GPT-Generated Unified Format**, though some sources also say it loosely means **Gerganov's General Unified Format**.
- It's the **successor to GGML format**.
- Designed by the **llama.cpp community**, it’s a **self-describing file format** to make quantized LLM deployment easier.

Use case:

Stores model weights + **complete metadata** (tokenizer, vocab, rope params, etc.) so tools don’t need manual flags. Hugging Face and llama.cpp fully support GGUF.

1. What is GGUF?

GGUF stands for **Gerganov’s General Unified Format**.

It is a new binary file format introduced as a **replacement for GGML models** for efficient storage and loading of quantized large language models (LLMs), especially in the llama.cpp ecosystem. It standardizes how models are stored, along with all metadata, tokenizer, quantization details, etc., into **one portable file**.

2. Why GGUF?

GGUF solves key problems of previous formats like GGML:

Problem in GGML	GGUF Solution
Separate files for tokenizer and metadata	☒ Single file with everything packed
Not versioned properly	☒ Formal versioning and backward compatibility
Hard to inspect	☒ Human-readable metadata using gguf-tool
Lacked standard tokenizer info	☒ Tokenizer, vocab, and pre-tokenizer embedded
Limited support for new quantization types	☒ Native support for INT4, Q8_0, Q6_K, etc.

3. GGUF File Contents

Each .gguf file stores:

Component	Description
model parameters	Hidden size, head count, vocab size, etc.
tensor weights	Quantized tensors like Q/K/V, FFN
tokenizer config	Pre-tokenizer, vocab, tokenizer type

metadata Format version, model source, training notes

quantization type E.g., Q4_K, Q8_0, etc.

4. Supported Models

GGUF works best with models using the **Transformers architecture**, such as:

- **LLaMA** / LLaMA 2 / LLaMA 3
- Falcon
- Mistral
- BLOOM / BLOOMZ
- GPT-J / GPT-NeoX
- Phi-2
- StarCoder

5. GGUF Format Structure (Internals)

At a low level, GGUF is a **binary format** with these parts:

- [Header]
- └─ Magic Number: GGUF
 - └─ Version: v1 or v2
- [Metadata]
- └─ Model name, tokenizer info, quantization method
- [Tensors]
- └─ List of all quantized model weights
 - └─ Each tensor has: name, shape, data type, actual data
- [Tokenizer Info]
- └─ Tokenizer type (BPE, SentencePiece, etc.)
 - └─ Special tokens, pre-tokenizer config, etc.

6. GGUF Quantization Types Supported

Type	Bits	Notes
Q4_0	4-bit	Basic 4-bit weights
Q4_K	4-bit	Activation-aware 4-bit (used in AWQ)
Q5_0	5-bit	Balanced accuracy/speed
Q8_0	8-bit	Near FP16 accuracy
F16	16-bit	Half-precision
F32	32-bit	Full precision

7. Tooling & Ecosystem

Tool	Purpose
llama.cpp	Main runtime for .gguf models
convert.py	Script to convert Hugging Face → GGUF
gguf-tool	Inspect, edit, and compare GGUF files
text-generation-webui	UI-based serving of GGUF
KoboldCpp, llamafile	Other backends that support GGUF

8. Real-World Workflow

- # Step 1: Quantize using AutoGPTQ or AWQ
- # Step 2: Export to GGUF using Transformers + llama.cpp scripts
- # Step 3: Run on CPU/Mac/low-VRAM GPU using llama.cpp

llama.cpp --model model.gguf --prompt "Explain black holes."

9. Pros & Limitations

- ☑ Pros:
- Compact single file
 - Easy deployment across devices
 - Standardized for llama.cpp ecosystem
 - Compatible with many quant formats
- ⚠ Limitations:
- Mostly optimized for decoder-only (causal) transformers
 - Not natively supported by Hugging Face libraries (only conversion possible)

10. Future-Proofing

- GGUF is under **active development**. It's expected to:
- Add support for **speculative decoding**
 - Allow **multi-model packing** (e.g., Mixture of Experts)
 - Embed custom metadata for retrieval, agents, etc.

11. Practical Example with LLMs

What is RAM?

RAM is your **computer's main memory**

It stores:

- OS processes
- Program data
- Neural network model (if running on CPU)

Example:

- You open Chrome + — these live in RAM
- Model running on CPU → RAM used
- Python Program Execution → RAM used

What is CPU (Central Processing Unit)?

Role: The "brain" of your computer — executes general-purpose instructions.

Example:

- Running Python scripts
- Browsing the web

What is VRAM?

VRAM is **dedicated memory on your GPU**

Stores:

- Model weights
- Activations during forward/backward pass
- Gradients (during training)
- Images / tensors (during inference)

Used by GPU (like NVIDIA, AMD)

Example:

- Model running on GPU (e.g. LLaMA) → weights loaded in **VRAM**
- If VRAM is low → model crashes (OOM)

What is GPU (Graphics Processing Unit)?

- **Role:** A highly parallel processor optimized for graphics & matrix/tensor operations.

Example:

- Running LLaMA or Stable Diffusion models
- Playing a high-end video game
- Video editing / 3D rendering

Feature	RAM	VRAM
Used by	CPU	GPU
Speed	Slower than VRAM	Extremely fast (optimized for tensors)
Application	Web, apps, CPU training	Deep learning inference/training on GPU
Capacity	8–64 GB typical	2–48 GB (depending on GPU)
Model hosted?	On CPU	On GPU
Quantization helps?	☒ Yes (smaller RAM needed)	☒ Yes (smaller VRAM needed)

Data Types with Memory Consumptions + Examples

◊ 8 bits = 1 byte

◊ float32 = 32 bits = 4 bytes per number

Data Type	Meaning	Size in Memory	Precision	Example Value	Precision Meaning	Usage in DL
float64	64-bit floating point (FP64) (optional)	8 bytes $8 \times 8 = 64$	Very High	3.141592653589793	~15–16 digits (scientific use)	Scientific computing, double precision
float32	32-bit floating point (FP32)	4 bytes $8 \times 32 = 32$	High	3.14159265	~7 decimal digits (e.g., 0.1234567)	Training & baseline
float16	16-bit floating point (FP16)	2 bytes $2 \times 8 = 16$	Medium	3.14	~3–4 decimal digits	Mixed precision training (NVIDIA AMP)
bfloat16	Brain float 16 (Google TPU)	2 bytes	Medium	3.14 (less mantissa)	~3–4 decimal digits	Training on TPUs
fp8	8-bit float (E4M3 or E5M2 variants)	1 byte	Low-Medium	~2.75 (approx)	~2–3 digits (approx)	NVIDIA Hopper GPUs, inference/training
nf4	Normal Float 4 (log-based float)	0.5 bytes	Very Low	0.5, 1.0, 2.0	Very coarse float values	bitsandbytes, GPTQ/AWQ

fp8	8-bit float (E4M3 or E5M2 variants)	1 byte	Low-Medium	~2.75 (approx)	~2-3 digits (approx)	NVIDIA Hopper GPUs, inference/training
nf4	Normal Float 4 (log-based float)	0.5 bytes	Very Low	0.5, 1.0, 2.0	Very coarse float values	bitsandbytes, GPTQ/AWQ
int8	8-bit integer	1 byte	Low	-128 to +127	No decimal (discrete values only)	Quantized inference
int4	4-bit integer	0.5 bytes	Very Low	-8 to +7	Very coarse discrete values	GPTQ, AWQ, extreme quantization
int32	32-bit integer	4 bytes	Exact	123456	Integer precision (no rounding)	Indexing, padding, tensor ops
bool	Boolean (True/False)	1 bit	None	True, False	Binary only (0 or 1)	Attention masks, binary outputs

Let's check some facts:

Let's say you have a neural net with **100 million parameters** (Weights and Biases)

Data Type Memory Required

float32 $100M \times 4B = 400 MB$

float16 $100M \times 2B = 200 MB$

int8 $100M \times 1B = 100 MB$

int4 $100M \times 0.5B = 50 MB$

Deep learning engineers **intentionally switch data types** based on use-case:

• **FP32** → Full training

• **FP16/BF16** → Mixed training

• **INT8/INT4** → Inference

Float32 → int8 / int4
 400 MB → 100 / 50 MB

3. What is Precision in Deep Learning?

Precision means:

"How exact, detailed, and accurate a number can be stored"

"Precision means how finely a number is stored."

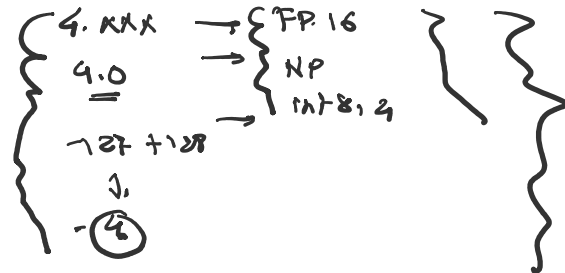
Precision Levels – Real-World Value Examples

"The more decimal values a number can store, the higher the precision."

"Float32 offers high precision but consumes more memory. INT8 offers lower precision, but it's much faster."

1. Mathematical Value / PI Examples

Value	Precision Level
3.14159265	High precision (float32)
3.14	Low precision (float16)
3	Low precision (int8)



2. Currency Example (e.g. Dollar \$ / ₹ Rupee)

Value	Precision Type
₹87.5793	High precision (float32-like)
₹87.57	Medium precision (float16-like)
₹87	Low precision (int8/int4-like)

3. Weight Measurement Example

Value	Precision Type
60.197 kg	High precision
60.2 kg	Medium precision
60 kg	Low precision

Technical View:

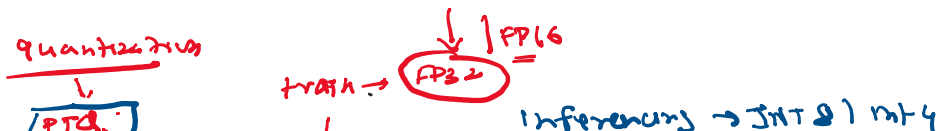
- float32 = 32 bits
 - Can store very fine decimal values (like 0.00000001)
- int8 = 8 bits
 - Can store only whole numbers from -128 to 127
 - No decimal → Low precision

So,

- float32 is high precision
- int8 is low precision

Why Does Precision Matter?

- During training, we need high precision to learn small gradients and improve accuracy.
- During inference, we can use low precision (e.g. int8), because we only need fast predictions — even if they are slightly less accurate.



$$\text{Quant} \leftarrow \text{FP32} \rightarrow \text{INT8}$$

Format	Bit Width	Used For	Precision Level	Memory	Speed	Accuracy (Numerical)
--------	-----------	----------	-----------------	--------	-------	----------------------

FP32	32-bit	Full precision training	◇ High (~7 decimal digits)	High	Slow	☑ Best
FP16	16-bit	Mixed precision training	◇ Medium (~3–4 decimal digits)	Medium	Faster	▲ Slight drop
INT8	8-bit	Most common for quantization	○ Low (no decimals, -128 to +127)	Low	Fastest	▽ Small accuracy loss
INT4 / NF4	4-bit	Research, LLM compression (bnb/GPTQ)	▽ Very Low (coarse discrete/float values)	Very Low	Ultra-Fast	✗ Moderate accuracy loss
FP8	8-bit	NVIDIA Transformer inference (H100)	◇ Low-Medium (~2–3 decimal digits)	Low	Fast	▲ Slight drop

Additional info:

Precision ≠ Accuracy — What Does It Mean?

◇ Precision

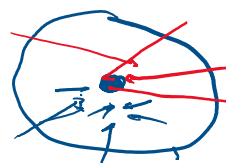
"How finely or in how much detail a number is represented or stored."

- Related to **data format** (float32, int8, etc.)
- Example:
 - 60.123456 → **high precision**
 - 60 → **low precision**

◇ Accuracy

"How close is the prediction/output to the correct/true value."

- Related to **model performance**
- Example:
 - Actual label = 1, model predicts 0.97 → **high accuracy**
 - Predicts 0.4 → **low accuracy**



Real-Life Analogy:

Imagine you're throwing darts 🎯

- **Precision** = How tight your dart group is (clustered close together)
- **Accuracy** = How close they are to the bullseye

You can have:

- 🎯 High precision, low accuracy: all darts tightly grouped but far from target
- 🎯 Low precision, high accuracy: darts scattered but average hits the center

Best: high precision + high accuracy

Key Point:

A model can be low precision but high accuracy

Example:

- You quantize a model from float32 to int8 (precision ↓)
- But model still gives **correct output** (accuracy maintained)
- So, **low precision ≠ low accuracy**

4. Formula of Quantization and Dequantization

Quantization Formula (Uniform Linear Quantization)

Quantization (Float → Int)

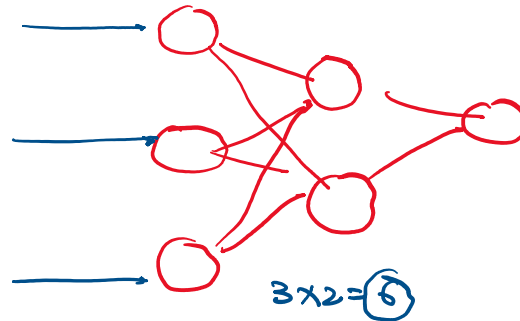
$$x_q = \text{round}\left(\frac{x}{\text{scale}}\right) + \text{zero_point}$$

Where:

- x = original float value
- x_q = quantized integer (e.g., int8)
- scale = how much range one int step covers
- zero_point = which integer represents 0.0 in float
- $\text{round}()$ = nearest integer

Dequantization (Int → Float)

$$x = (x_q - \text{zero_point}) \times \text{scale}$$



$$\begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \\ w_{31} & w_{32} \end{bmatrix} \rightarrow \begin{bmatrix} 0.673 & 0.401 \\ 0.772 & 0.62 \\ 0.12 & 0.539 \end{bmatrix}$$

FP → 32 → 1x8

H₁ 17₂

This is used during inference to convert quantized weights back to usable float values inside the neural network.

What is the Goal of Quantization?

To store **real numbers** (like float32: 1.23, -0.85, 2.77...) in **less memory** using small integers such as -128 to +127 or 0 to 255.

Math behind quantization:

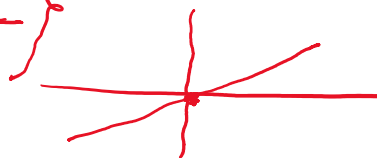
- Scale & Zero-point
- Clipping and rounding
- Range mapping

WFB → float-32

What's the Problem?

Float values can sometimes fall into different ranges:

- Sometimes in range like $[-2.0, +2.0]$
- Sometimes only positive, like $[0.5, 2.3]$
- Sometimes unbalanced like $[-5.7, 8.3]$



To solve this — we use **scale** and **zero_point**:

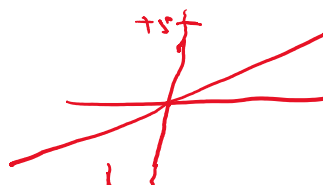
$$x_q = \text{round}\left(\frac{x}{\text{scale}}\right) + \text{zero_point}$$

Symmetric vs Asymmetric Quantization Explained

◇ Symmetric Quantization:

Definition:

- $\text{zero_point} = 0$



Definition:

- zero_point = 0
- You center the data around 0
- You assume the data is in range like $[-X, +X]$
- **Quantization range:** $[-128, +127]$ (for INT8)

Tensor Range Quant Range Quant Type Zero Point

$[-2.0, +2.0]$	$[-128, 127]$	Symmetric	0
$[0.5, 2.5]$	$[0, 255]$	Asymmetric	Non-zero
$[-3.5, 8.2]$	$[0, 255]$	Asymmetric	Non-zero

Example:

Let's say your model weights lie between $[-2.0, +2.0]$
→ Equal spread of negative and positive values
→ Center = 0

☒ So here, **symmetric quantization** is the best choice

$$x_q = \text{round}\left(\frac{x}{\text{scale}}\right)$$

zero_point = 0 (no shifting needed)

Example For Symmetric :

Let's say:

- Float value $x=1.23$
- Quant range = INT8 = $[-128, +127]$
- Min/Max of tensor = $[-2.0, +2.0]$

Step 1: Calculate scale

$$\text{scale} = \frac{\text{max} - \text{min}}{q_{\text{max}} - q_{\text{min}}} = \frac{2.0 - (-2.0)}{127 - (-128)} = \frac{4.0}{255} \approx 0.0157$$

Step 2: No zero point

Since symmetric quantization
zero_point=0

Step 3: Quantize

$$x_q = \text{round}\left(\frac{x}{\text{scale}}\right) = \text{round}\left(\frac{1.23}{0.01575}\right) \approx \text{round}(78.1) = 78$$

☒ 78 is the **quantized integer representation** of float 1.23

Step 4: Dequantize

$$x = x_q \times \text{scale} = 78 \times 0.01575 \approx 1.229$$

You get back approximate float: **~1.229** very close to **1.23**

You get back approximate float: ~ 1.229 very close to 1.23

Asymmetric Quantization:

Definition:

- zero_point $\neq 0$ (non-zero shift)
- Used when data is **not centered around 0** (e.g., only positive values like [0.5, 2.5])
- **Quantization range:** [0, 255] (e.g., uint8)

$$x_q = \text{round}\left(\frac{x}{\text{scale}}\right) + \text{zero_point}$$

Example:

If model weights are only positive: [0.5, 2.5]

Center = 1.5 (not 0)

✓ So in this case, **asymmetric quantization** is preferred

zero_point $\neq 0$ — this adjusts the mapping so the minimum float value aligns near 0 in the quantized space

Example For Asymmetric :

Parameter	Value
Float value x	1.23
Quant range	[0, 255] → using uint8
Min/Max of tensor	[0.5, 2.5]

Step 1: Calculate scale

$$\text{scale} = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}} = \frac{2.5 - 0.5}{255 - 0} = \frac{2.0}{255} \approx 0.00784$$

Step 2: Calculate zero_point

$$\text{zero_point} = \text{round}\left(-\frac{r_{\min}}{\text{scale}}\right)$$

$$\text{zero_point} = \text{round}\left(-\frac{0.5}{0.007843}\right) = \text{round}(-63.75) = -64$$

(0, 255)

Since zero_point cannot be negative for uint8, we clip it: ← clipping

$$\text{zero_point} = \max(0, \min(255, -64)) = 0$$

Step 3: Quantize x = 1.23

$$x_q = \text{round}\left(\frac{x}{\text{scale}}\right) + \text{zero_point} = \text{round}\left(\frac{1.23}{0.00784}\right) + 0 = \text{round}(156.887) \approx 157$$

Step 4: Dequantize

$$x = (x_q - \text{zero_point}) \times \text{scale} = (157 - 0) \times 0.00784 \approx 1.23$$

Parameter	Value
Float value x	1.23 ← weight

Parameter	Value
Float value x	1.23 ← <i>weight</i>
Quant range	[-128, 127] → INT8
Tensor Min/Max	[0.5, 2.5]

◆ Step 1: Compute Scale

$$\text{scale} = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}}$$

Substitute:

$$\text{scale} = \frac{2.5 - 0.5}{127 - (-128)} = \frac{2.0}{255} = 0.007843$$

◆ Step 2: Compute Zero Point

$$\text{zero_point} = \text{round} \left(-\frac{r_{\min}}{\text{scale}} \right)$$

$$\text{zero_point} = \text{round} \left(-\frac{0.5}{0.007843} \right) = \text{round}(-63.75) = -64$$

◆ Step 3: Quantize float value x = 1.23

$$x_q = \text{round} \left(\frac{x}{\text{scale}} \right) + \text{zero_point}$$

$$x_q = \text{round} \left(\frac{1.23}{0.007843} \right) + (-64) = \text{round}(156.8) - 64 = 157 - 64 = 93$$

Most quantized models use INT8 or INT4, not UINT8. And we rarely check tensor distribution (centered or not)

Hardware Preference — INT8/INT4 is Standard

- Almost all modern accelerators (CPU, GPU, NPU, TPU) are optimized for **signed int8/int4**:
 - **AVX2 / AVX-512 (Intel)** → supports int8
 - **NVIDIA TensorRT / Ampere / Hopper** → optimized for int8, int4
 - **ARM / Qualcomm chips (mobile)** → expect int8
 - **LLMs (GPTQ, AWQ)** → heavily use int4 / int8
 - **uint8** lacks widespread hardware support.

Framework Defaults

- Libraries like torch.quantization, onnxruntime, transformers, etc. default to **symmetric int8**.
- Even when using asymmetric quantization, they **still keep data in int8** but shift it with non-zero zero_point.
- ⚠️ So even in "asymmetric quantization", **they use int8, not uint8** most of the time.

Simplicity and Memory Alignment

- int8 has a **symmetric range** (-128 to +127) which makes it easy to:
 - Zero-center activations
 - Align quantized data in memory
- Frameworks like GPTQ, bitsandbytes use **quantized linear algebra** that works with int8/int4, not uint8.

Real-world model weights are mostly symmetric

- Most transformer weights like:
 - Query / Key / Value
 - Feedforward layers
 - Conv filtersAre centered around 0
- So **symmetric quantization using int8 works very well**
Only some embeddings / biases / activations may be asymmetric — handled with **per-channel quant + offset**

Term	Meaning
Quantization Error	Difference between real and quantized-dequantized value
Calibration	Forward-passing real data to compute min/max ranges for quantization

Per-Tensor Quantization

☑ Definition:

- The **entire tensor** (e.g., full weight matrix) shares a **single scale and zero-point**.

Use Case:

- Simpler and faster.
- Often used for **activations**.
- But less accurate, especially if different channels have different value ranges.

Example:

Imagine a weight tensor of shape (4, 3):

```
[
  [0.1, 0.2, 0.3],
  [1.5, 2.0, 1.8],
  [0.05, 0.07, 0.08],
  [3.0, 2.5, 3.2]
]
```

In **per-tensor**, we calculate **1 scale + 1 zero-point** for all values above.

Even though the ranges vary across rows, they all share the same quantization formula.

2. Per-Channel Quantization

☑ Definition:

- Each **channel** (e.g., each row in weight matrix) gets its **own scale and zero-point**.

Use Case:

- Used **mostly for weights** in **CNNs, LLMs**, etc.
- More accurate.
- Especially useful when each channel has a different range of values.

Example:

Using same matrix:

```
[
  [0.1, 0.2, 0.3],    # Channel 1 → small range → scale1
  [1.5, 2.0, 1.8],    # Channel 2 → medium range → scale2
  [0.05, 0.07, 0.08], # Channel 3 → tiny range → scale3
  [3.0, 2.5, 3.2]     # Channel 4 → large range → scale4
]
```

In **per-channel**, we compute **4 different scales** and **4 zero-points** (one for each row).

This allows fine-tuned quantization and **reduces loss in accuracy**.

Feature	Per-Tensor	Per-Channel
Granularity	Single scale/zero-point	One scale/zero-point per channel
Speed	Faster (simpler)	Slightly slower
Accuracy	Lower	Higher
Use Case	Activations	Weights (especially in LLMs, CNNs)
Preferred for	Small models or quick inference	High accuracy + transformer models

☑ In Practice (LLMs & CNNs):

- **LLMs** mostly use:
Symmetric + Per-Channel Quantization for weights (efficient & accurate).
- **CNNs/activations** might use:
Asymmetric + Per-Tensor Quantization for dynamic range handling.

In Transformers (like LLaMA, BERT):

- **Query, Key, Value, and FFN weights** are **quantized per-channel** (row-wise) to preserve model behavior.
- This is usually **symmetric + per-channel**.

Summary

- **Per-tensor** = simple, fast, but may lose accuracy if value distribution varies.
- **Per-channel** = precise, handles varying ranges well — best for large models and weights.

Bonus:

What is Quantization Error?

Quantization error means:

The small difference between original float value and quantized (int) value after you quantize and then dequantize it.

Why it happens?

Because you're trying to represent a high-precision value (e.g., 3.14159265) using a **low-precision integer** (e.g., int8 only has 256 possible values).

Example:

Suppose you quantize a float:

- Original: $x = 1.23$
- After quantization + dequantization: $x' = 1.21$

Then:

$$\text{Quantization Error} = x - x' = 1.23 - 1.21 = 0.02$$

Implication:

- Too much error → model accuracy drops
- That's why quantization must be done **carefully** (especially for LLMs and sensitive models)

What is Calibration?

Calibration is:

The process of analyzing real data to find accurate min/max values of tensors for quantization.

It's used in **Post Training Quantization (PTQ)**.

How?

- During calibration, you **pass a few batches of real input data** (no training, just forward pass)
- This helps the quantizer figure out:
 - Tensor ranges (min/max)
 - Which layers can use symmetric or asymmetric quant
 - Best scale & zero-point

Why calibration is important:

- Without calibration, wrong min/max values may cause **overflow or clipping**
- Model output becomes garbage!

```
model.eval()
```

```
model.qconfig = torch.quantization.get_default_qconfig('fbgemm')
```

```
torch.quantization.prepare(model, inplace=True)
```

```
# 🖱️ Calibration step:
```

```
for batch in calibration_data:
```

```
    model(batch) # Just forward pass
```

```
torch.quantization.convert(model, inplace=True)
```

- **Zero-point is always 0**

This means the quantized value 0 corresponds to the real-world value 0.

- **Range is centered around 0**

For example, in int8, the range is typically [-127, 127].

- **Used when data is centered around zero** (like weight tensors).
- **Simpler math**, better for hardware acceleration (e.g., matrix multiplies).
- **Preferred in LLMs**, especially with **per-channel quantization**.

- **Zero-point is NOT zero**

Used when the data distribution is not centered around 0 (e.g., activation outputs).

- **Range is from [min, max]**

e.g., [0, 255] for uint8 or mapped real-world values.

- More flexible for **non-centered distributions**, like ReLU activations.
- Adds extra steps in inference math (subtracting zero-point), but works better for **diverse**

value ranges.

5. Type of Quantization

PTQ Post-Training Quantization

- SQ STATIC Quantization
- DQ Dynamic Quantization

QAT Quantization-Aware Training

① training
② $w \& b \rightarrow \text{quantize}$ ← Formula

PTQ = Post-Training Quantization

The model has already been trained, and now we are quantizing it (without retraining).

QAT = Quantization-Aware Training

"The model is trained with quantization in mind — it learns to handle quantization during training itself."

Intuition First: Easy Analogy

Imagine you're training a cricket player to bat:

🔪 **PTQ: "Train with perfect bat, play with cheap bat later"**

- You train with a high-quality bat (float32 weights)
- But in the match, you suddenly give him a plastic bat (int8)
- He might still perform okay, but maybe not perfect — he never practiced with this bat!

🔪 **QAT: "Train and play with the same cheap bat"**

- You train the player from day one with the exact bat (int8) he'll use in matches
- So he learns how to handle its limitations
- Outcome: more consistent performance

training quantization

FP → int8

PTQ → FP → int8

inferencing

SCALAR
ALL
double

PTQ → QAT → QAT (wrap up) → OIP

What is Calibration Data? (Simple Words)

It's just some sample input data that you run through the model to measure the activation range (min/max).

Example:

Calibration Data = Just dummy inputs used to find out what activation range the model typically produces.

Let's say your model's hidden layer gives values between (-3.5 to +5.7)

Now You'll use this range to map it into INT8 format (-128 to +127)

Feature	PTQ	QAT
When it's applied	After model training is done	During training (starts with float model)
What it does	Quantizes weights and activations after training	Simulates quantization during training
Model learning aware?	No — model never saw int8 during training	Yes — model learns to adapt to int8 errors
Accuracy impact	Slight accuracy drop (1–5%)	Minimal or no accuracy drop
Use-case	For fast compression after training	For production-grade quantization (e.g. edge)
Training time	No retraining needed — very fast	Retraining required — slower
Toolkits	torch.quantization.prepare/convert, GPTQ, BnB	prepare_qat, AutoQAT, TensorRT-QAT
Complexity	Easy to implement	More complex (fake quant, training config)

Complexity

Easy to implement

More complex (fake quant, training config)

Model	Trained On	Predicts like
Float32	Full precision	$3.56 + 1.82 = 5.38$
PTQ	Float precision, then INT8 inference	$3 + 2 = 5$ ☒ (approx. ok)
QAT	Trained with INT8 simulation	$3 + 2 = 5$ ☒ (learns best rounding)

Aspect	PTQ	QAT
Training needed	No	Yes
Accuracy loss	Some (1–5%)	Minimal or none
Speed	Very fast	Slower (retraining needed)
Ease of use	Easy	Moderate to hard
Activation quant?	Optional or estimated	Accurate (learned)
Real int8 backend?	Yes after convert()	Yes after convert()
Toolkits	torch.quantization, GPTQ, bnb	prepare_qat, convert in PyTorch

Scenario	Best Option
Compress model for faster inference	PTQ
Deploy on edge/mobile (high accuracy needed)	QAT
Need quick model size reduction	PTQ
You're training from scratch or fine-tuning	QAT
LLM inference only	PTQ (GPTQ, AWQ)

What happens inside a model:

- **Weights & Biases** → These are learnable parameters of the model.
- **Activation values** → These are the outputs from each hidden layer after applying the activation function.
- **Forward Pass** → The input passes through the layers to produce the output. (This is what happens during inference.)
- **Backward Pass** → Gradients are calculated and used to update the weights. (This happens only during training.)

Type	Quantized Parameters	Calibration	Common Name
Static PTQ	Weights ☒ + Activations ☒	☒ Yes	Static Quantization
Dynamic PTQ	Weights ☒ only	☒ No	Dynamic Quantization

Normal or Partial PTQ = Weight only Quant

Static PTQ = Calibration = Activation + Weight Quant

Dynamic PTQ = No Calibration = Weights are pre-quantized (saved as INT8), but activations are quantized temporarily during inference time (on-the-fly), not permanently — no calibration required."

QAT = Training time per quantization ka practice

Type	Quantize Weights	Quantize Activations	Needs Calibration	Training Involved
------	------------------	----------------------	-------------------	-------------------

Type	Quantize Weights	Quantize Bias	Quantize Activations	Needs Calibration	Training Involved
PTQ (Static)	☒	☒	☒	☒ Yes	☒ No
PTQ (Dynamic)	☒	☒	☒ (done at runtime)	☒ No	☒ No
QAT	☒	☒	☒	☒ No	☒ Yes (full training)

Feature	Dynamic Quantization	Static Quantization
Weights	INT8 (pre-quantized)	INT8 (pre-quantized)
Activations	Quantized on-the-fly	Quantized beforehand using calibration
Calibration	☒ Not needed	☒ Needed
Accuracy	Moderate	Better
Speed	Fast	Faster (usually)

PTQ (Post-Training Quantization)

The model is already trained (in FP32), and then you apply quantization afterward.

Types:

- **Weights Quantization** — ☒ Done
- **Bias Quantization** — ☒ Done (usually)
- **Activation Quantization** — ☐ It depends:
 - If you use **Static Quantization (SQ)** → ☒ Activations are quantized
 - If you use **Dynamic Quantization (DQ)** → ☒ Activations are *not* pre-quantized (they're quantized on-the-fly during inference)

STATIC Quantization (SQ)

☒ A type of PTQ where **both weights and activations** are quantized.

A **calibration dataset** is used to calculate the min/max values of activations.

Process:

- You perform a **forward pass** of the model on a small calibration dataset.
- For each layer, you **collect the min/max range of activation values**.
- Then, you use that to **pre-quantize activations** into **INT8 format**.

Static Quantization = requires calibration + quantizes both weights and activations.

DYNAMIC Quantization (DQ)

☒ Also a type of PTQ, but **lighter** and faster to apply.

- Only **weights** are quantized (to **INT8**)
- **Activations remain in FP32** and are quantized **on-the-fly** during inference

Dynamic Quantization =

No calibration required,

Only weights are quantized,

Activations are dynamically quantized during inference.

QAT (Quantization Aware Training)

During training, the model is made to "*feel*" that it will be quantized later.

Inside QAT:

Forward Pass:

- Quantization is **simulated** (rounded off to INT8) during the forward pass
- → The model **learns to handle quantization error**

Backward Pass:

- **Backpropagation** happens on actual **FP32 weights**,
- but the **gradient is computed w.r.t. quantized values**
- This technique is called the "**Straight-Through Estimator (STE)**"

QAT =

Training + Simulate quantization during forward pass +
Backward pass on FP32, but loss comes from quantized output.

1 STATIC Quantization

Summary:

Weights and activations — both are quantized **ahead of inference** using a **calibration dataset**.

How it works:

- Model is trained in float32
- You provide **a few batches of real data** (calibration)
- It computes min/max of **weights & activations**
- Then it **quantizes both** statically

Used When:

- You need full optimization
- Target device supports full INT8 pipeline
- You can afford calibration step

Real-Life Analogy:

You test every camera setting before a shoot, lock all values (brightness, exposure) — ready to shoot.

Pros:

- Very optimized (speed + memory)
- Works well on mobile (qnnpack) & edge (fbgemm)

Cons:

- Needs calibration data
- Slight risk of poor accuracy if calibration is bad

2 DYNAMIC Quantization

Summary:

Weights are quantized ahead of time, but **activations are quantized on-the-fly** at runtime.

How it works:

- Weights are stored as int8
- Activations are converted to int8 **during inference**
- No calibration needed

Used When:

- You want **quick quantization** with minimal effort
- Deployment is CPU-only (e.g., AWS CPU, Raspberry Pi)

Real-Life Analogy:

You keep your camera in auto-mode. It adjusts lighting **dynamically** at each moment.

Pros:

- No calibration needed
- Easy to apply to pre-trained models (especially LLMs)

Cons:

- Slightly less optimized than static

- Works best for **fully connected layers**, not CNNs

3 QUANTIZATION-AWARE TRAINING (QAT)

Summary:

Simulates quantization **during training** so model learns how to handle low-precision.

How it works:

- Insert **fake quant/dequant ops** during training
- Backprop still uses float32, but forward uses simulated int8
- After training, you convert to real int8 model

Used When:

- Accuracy is super critical
- Deploying on tight-resource devices (edge, phones)

Real-Life Analogy:

You train a cricketer using a plastic bat from Day 1 — so he masters it, even in real matches.

Pros:

- Best accuracy
- Learns to handle quantization errors

Cons:

- Slower (requires retraining)
- More complex to set up

6. LLM Advance Quantization Introduction

Term Meaning & Clarification

FP Floating Point

INT Integer (e.g., INT8, INT4 used for quantization)

PTQ Post-Training Quantization — quantize a model **after full training** without re-training

QAT Quantization-Aware Training — simulate quantization during training to **preserve accuracy**

GPTQ Originally stands for "**Gradient Post-Training Quantization**". It is a **specific PTQ technique** designed for transformers. People confuse it with "Generative Pretrained Transformer Quantization", but that is incorrect.

AWQ Activation-aware Weight Quantization — focuses on **scaling weights w.r.t activations** for better accuracy in INT4/INT8

GGM Georgi Gerganov's Machine Learning library for inference — supports CPU/GPU/backends

L

GGUF Gerganov's General Unified Format — successor to GGML binary formats. Used to store **quantized models** (e.g., LLaMA, Mistral) for use in tools like llama.cpp.
! Some people say it stands for "GPT-Generated Unified Format" but that's a **backronym**, not official.

Extra Notes (for depth):

- GPTQ = *Post-Training Quantization specifically for Transformers*
- GGUF = *Efficient model file format used by llama.cpp ecosystem*
- GGML = *Inference engine that consumes GGUF or older GGML format for CPU/low-resource inference*

One-Line Summary (Crisp & Final)

- **PTQ, QAT** = *Quantization techniques* (how quantization is done).
- **GPTQ, AWQ** = *Quantization algorithms/libraries* (enhanced PTQ methods for LLMs).
- **GGUF** = *Model file format* (used for saving/exporting quantized models).
- **GGML** = *Inference runtime* (engine that runs quantized models like GGUF).

PTQ → GPTQ → GGUF → llama.cpp or GGML

Refined Example Flow (End-to-End):

1. You have a **LLaMA-3 8B model** with **FP16 weights**.
2. You perform **quantization using GPTQ** (which is a type of **Post-Training Quantization**).
3. You **export the quantized model in GGUF format** (which is compatible with llama.cpp).
4. Finally, you **run the model using llama.cpp or a GGML-based engine** — on **CPU or Mac**, no GPU required.

♦ GGML

- GGML = **low-level C library** for **tensor operations + inference runtime**.
- Optimized for **CPU inference** (also has support for Metal on macOS, and some GPU offloading).
- Used as the **core backend** in projects like llama.cpp, whisper.cpp, etc.
- Files like ggml-model-q4_0.bin are indeed **quantized model weight files** stored in GGML's custom format.

♦ llama.cpp

- llama.cpp = **C++ implementation** built on top of GGML/GGUF for running LLaMA-family models locally.
- Provides **CLI + C API + Python bindings** (via llama-cpp-python).
- Supports **quantized GGML/GGUF model formats** → efficient inference on CPU/Mac/low-VRAM GPU.
- "High-performance C++ wrapper" is almost right — actually it's more of a **full C++ project** that *uses GGML/GGUF as backend*. Not just a wrapper.

Quantization Libraries:

- **GPTQ (AutoGPTQ)** — Fastest for 4-bit quantization (Hugging Face compatible).
- **AWQ** — 4-bit quantization with activation-aware optimization (better accuracy).
- **bitsandbytes** — Enables INT8 / 4-bit loading directly in Hugging Face models.

Model File Format:

- **GGUF** — Efficient file format for storing quantized models (used with llama.cpp).

Inference Runtimes:

- **GGML** — C-based lightweight inference engine (runs quantized models on CPU/Mac).
- **llama.cpp** — Wrapper around GGML with CLI + API interface for easy use.

GPU/Server Deployment:

- **TensorRT-LLM, vLLM, ONNX Runtime** — High-throughput GPU inference on server setups.

Quantization Workflow Sequence

1. **You perform quantization** — using any of the following methods:
 - **PTQ** (Post Training Quantization)
 - **QAT** (Quantization Aware Training)
 - **GPTQ** (Gradient-based PTQ for Transformers)
 - **AWQ** (Activation-aware Weight Quantization)
2. **Then you save the quantized model in a specific format, such as:**

- **GGUF** (popular for the LLaMA + llama.cpp ecosystem)
 - **safetensors** (secure tensor format used in Hugging Face Transformers)
 - **ONNX** (used for inference with ONNX Runtime)
 - **TensorRT engine** (for NVIDIA-optimized inference)
3. And finally, you need an engine to run the saved format, such as:
- **llama.cpp** (for GGUF models) **GGML** (older format, now replaced by GGUF)
 - **vLLM** (used with safetensors for high-throughput inference)
 - **TensorRT-LLM** (optimized for INT4/INT8 inference on NVIDIA GPUs)

Tool / Format	Short Refined Explanation
GPTQ	Post-Training Quantization method (weight-only) for LLMs. Hugging Face + AutoGPTQ integration allows fast 4-bit inference on GPUs. Good balance of speed & accuracy.
AWQ	Activation-aware Weight Quantization . Focuses on minimizing accuracy drop in 4-bit models by preserving important activation patterns. Often slightly better quality than GPTQ in same bit-width.
bitsandbytes	Popular Hugging Face-compatible library for 8-bit & 4-bit loading (LLM.int8(), nf4). Very easy for quick memory savings without retraining.
GGML	C-based inference runtime for quantized models. Optimized for CPU and lightweight GPUs. Backbone for llama.cpp(ggml-model-q4_0.bin)
llama.cpp	High-performance C++ wrapper for GGML/GGUF models. Offers easy CLI tools + C/C++/Python APIs for running LLMs locally without heavy dependencies.
GGUF	Quantized model file format used in llama.cpp ecosystem. Stores weights efficiently in various bit-widths for fast CPU/Mac/low-VRAM GPU inference. Successor to GGML's .bin format.
TensorRT-LLM	NVIDIA library for optimized GPU inference with quantized or FP16/BF16 models. Great for production servers.
vLLM	High-throughput LLM serving engine. Optimized for batching, paged attention, and multi-GPU scalability.
ONNX Runtime	Cross-platform runtime for models exported in ONNX format . Supports CPU, GPU, TensorRT execution with quantization.

Can We Do PTQ/QAT on LLMs?

Yes — **Post-Training Quantization (PTQ)** and **Quantization-Aware Training (QAT)** are both applicable to LLMs, but the process is significantly different from small-scale models like CNNs or MLPs.

PTQ on LLMs

In MLPs (simple case):

scale = (rmax - rmin) / (qmax - qmin)
 zero_point = round(-rmin / scale + qmin)

Done on small tensors (e.g., linear layers)

Can use torch.quantization directly

In LLMs:

- Same core formula, but with **huge tensors**.
- Quantization is done:
 - **Layer-wise** or **per-channel**
 - On weights like **Q/K/V, Feed Forward (FFN)**, etc.
- Requires a **small calibration dataset** (~50–200 samples).
- Very sensitive to precision loss

Tools for PTQ on LLMs:

- GPTQ (Gradient-aware PTQ)
- AWQ (Activation-aware Weight Quantization)
- AutoGPTQ (wrapper around GPTQ + Hugging Face)
- bitsandbytes (8-bit & 4-bit loaders)

QAT on LLMs

In MLPs (simple case):

- Use QuantStub/DeQuantStub to simulate quant ops.
- **Fake quantization** during training.
- Backprop through **quantization ops** with minimal overhead.

In LLMs:

- Much more **complex and expensive**:
 - You simulate quant ops using **STE (Straight-Through Estimator)**.
 - Requires **full or partial retraining** of the model.

- Requires **multi-GPU** or **TPU setups** (very compute-heavy).

Tools for QAT on LLMs:

- torchao (PyTorch QAT backend)
- Intel Neural Compressor
- NVIDIA TensorRT-LLM (QAT support)
- OpenVINO (QAT & PTQ optimizations)
- HQQ (High-Quality Quantization + QAT)
- SmoothQuant (can be combined with QAT)

Component	Notes
◊ Activation quantization	Challenging due to softmax, layernorm, attention
◊ Weight quantization	Done on Q/K/V, FFN, etc., often with per-channel granularity
◊ Techniques	Dynamic, static, and fused quantization techniques are often combined

Aspect	MLPs	LLMs (e.g., GPT, LLaMA)
PTQ	Easy (torch.quantization)	Layer-wise, needs calibration, uses GPTQ/AWQ
QAT	Light training needed	Heavy retraining with STE, uses advanced toolkits
Activation Handling	Simple	Complex (softmax, attention layers)
Tools	Built-in torch	GPTQ, AWQ, bitsandbytes, torchao, TensorRT-LLM
Hardware Need	Single GPU	Multi-GPU/TPU for QAT

7. What is GPTQ?

- GPTQ (Gradient Post-Training Quantization) is an advanced Post-Training Quantization (PTQ) method specifically designed for Large Language Models (LLMs) like LLaMA, GPT, Mistral, Falcon, etc.
- It quantizes model weights to low precision (e.g., INT4) after training, without any fine-tuning, while still preserving the original model's accuracy as much as possible.

Key Characteristics

- Advanced PTQ method using gradient-based optimization
- Designed especially for Transformers and LLMs
- Targets INT4/INT8 quantization for model compression and faster inference
- Does not require retraining the model

What Does "GPT" Mean in GPTQ?

Despite "GPT" in the name, GPTQ actually stands for:
Gradient Post-Training Quantization

Which means:

Use gradients (second-order information: Gradient of gradient — Hessian matrix) during the quantization process to minimize quantization error — without retraining the model.

Intuition Behind GPTQ (Gradient Post-Training Quantization)

Typical PTQ:

- Post-Training Quantization (PTQ) usually just replaces full-precision weights with low-precision (e.g., INT4 or INT8) equivalents.
- It focuses on minimizing the difference between original and quantized weights.
- But this doesn't always ensure that the model's final output remains close to the original.

GPTQ is smarter:

1. Tracks the error that quantization introduces in each layer — not just at the weight level, but also how it affects the model's output.
2. Uses second-order information like gradients or Hessian approximations to understand the sensitivity of weights — i.e., which weights are more important for the model's performance.
3. GPTQ then selectively quantizes weights in a way that minimizes the final output error, not just weight-wise difference.

Concept	GPTQ Behavior
Type of Quantization	Post-Training Quantization (PTQ) – no model retraining required(Transformer-based LLMs (e.g., LLaMA, Falcon, GPT, BLOOM))
Quantized Precision	Typically INT4, occasionally INT3 or INT2 for extreme compression
Target Components	Linear layers within Transformer blocks – including Q/K/V, FFN, etc.
Core Idea	Quantize weights layer-by-layer using second-order optimization to minimize output error, not just weight error
Accuracy	High — performance remains very close to the original FP16 model High (if properly calibrated and quantized)
Speed	Much faster than QAT — ideal for post-training model deployment Very fast inference (esp. with ExLlama, llama.cpp, etc.)
Calibration Data	Required: around 50–200 representative text samples for accurate quantization

Quantized GPTQ models can be saved in various formats depending on your deployment pipeline:

Format	Description
.safetensors	Secure & efficient format used by Hugging Face ecosystem (e.g., AutoGPTQ)

.gguf Converted format for use with llama.cpp (fast inference on CPU/Mac)
.pt or custom PyTorch checkpoint format or internal formats for specific libraries

Library / Tool	Description
AutoGPTQ	Hugging Face compatible wrapper around GPTQ; works with Transformers easily.
GPTQ-for-LLaMA	Original low-level repo by Yao Fu et al. implementing GPTQ from scratch.
ExLlama & ExLlamaV2	Fast inference engines for GPTQ quantized models (especially on CUDA GPUs).
llama.cpp	Supports running GPTQ quantized GGUF models (CPU/Mac inference). Hugging Face Transformers, bitsandbytes

Simple Terms:

- **Standard PTQ** just compresses numbers layer-wise using **min/max scaling**.
- **GPTQ** is smarter: it checks **how much quantizing a weight will affect model outputs**, and avoids high-impact quantization errors using the Hessian and **greedy + compensatory logic**.

Real-Life Analogy:

- **Standard PTQ** = blindly shrink all clothes to medium size.
- **GPTQ** = check body shape, try fitting each shirt, and adjust the next one based on how the last one shrunk. Smart and adaptive!

GPTQ vs Standard PTQ: **Key Differences**

✅ Verified Comparison: Standard PTQ vs GPTQ (Advanced PTQ)		
Aspect	Standard PTQ (Basic)	GPTQ (Advanced PTQ)
Goal	Quantize weights to INT8/INT4	Same goal, but minimizes output error more precisely
Formula	$x_q = \text{round}\left(\frac{x}{\text{scale}}\right) + \text{zero_point}$ 👉 Simple scalar rounding based on min/max	$\min_{\hat{W}} \ XW^T - X\hat{W}^T\ _F^2$ 👉 Matrix approximation to reduce output deviation
Level	Mostly tensor-wise or channel-wise	Always row-wise quantization (per weight vector)
Error Awareness	❌ No modeling of how quantization affects final output	✅ Uses Hessian (2nd-order gradients) to estimate the error introduced
Error Compensation	❌ No	✅ Yes — adjusts quantization to compensate future layer errors
Calibration Data	Optional (often skipped)	✅ Mandatory (~50–200 text samples) to compute statistical structure like $X^T X$
Math Behind	Basic linear scaling	Linear algebra: matrix decomposition + 2nd-order correction
LLM Use Case	❌ Rarely sufficient for Transformer ^{~4} LMs ↓	✅ Specifically designed for LLMs like LLaMA, GPT, Falcon, etc.

Core Idea Behind GPTQ

GPTQ treats quantization as a **matrix approximation problem**:

You're replacing the original full-precision weight matrix **W** with a quantized version **Ŵ (W-hat)**.

Objective:

Minimize the output difference caused by using **Ŵ instead of W**, i.e.

$$W \cdot X \approx \hat{W} \cdot X$$

Where:

- W = Original weight matrix
- $\hat{W}$ = Quantized weight matrix
- X = Input activations

Step-by-Step Mathematics of GPTQ

1. Model Layer Representation

Let:

- $W \in \mathbb{R}^{n \times d}$ be the full-precision weight matrix.
- $X \in \mathbb{R}^{b \times n}$ be the calibration input (b = batch size).
- The output of the linear layer:

$$Y = XW^T$$

2. Quantization Goal

We want a quantized version $\hat{W}$ ($\hat{W}$) such that:

$$\text{Loss} = \|XW^T - X\hat{W}^T\|_F^2$$

Here:

- $\|\cdot\|_F^2$ is the **Frobenius norm** (sum of squared differences).
- The loss is minimized over **rows of W** , one at a time.

This is why GPTQ is **row-wise quantization**.

3. Hessian Matrix Approximation

To preserve the model's output better, GPTQ incorporates **second-order error correction** using the **Hessian matrix**:

- Approximate local curvature of the loss landscape w.r.t. weights.
- Hessian for each row i of W :

$$H_i = X^T X$$

This is a **covariance matrix** of the inputs. It tells us **which directions in W are more sensitive** to changes.

4. Blockwise Greedy Quantization

- Quantize weights **one at a time** in blocks.

- For each weight w_{ij} , choose quantized value $\hat{w}_{ij}$ that minimizes:

$$\text{Error Increase} \approx (w_{ij} - \hat{w}_{ij})^2 \cdot H_{jj}$$

If H is diagonalized, this is fast and local.

5. Error Compensation

After quantizing a weight w_{ij} , **update remaining weights** to compensate for the quantization error:

$$w_{ik} := w_{ik} - \frac{H_{jk}}{H_{jj}} \cdot (w_{ij} - \hat{w}_{ij})$$

This step is similar to **Low-Rank Approximation** where later rows compensate for errors introduced by earlier ones.

Step	Formula
Linear Output	$Y = XW^T$
Loss Function	$\ XW^T - X\hat{W}^T\ _F^2$
Hessian (Row-wise)	$H = X^T X$
Quant Error Estimate	$(w - \hat{w})^2 \cdot H$
Error Compensation	$w' = w - \text{correction}$

Calibration Dataset

- GPTQ requires a small set of **real inputs (50–200 samples)** to simulate real inference data.
- This helps compute XXX and hence HHH, making quantization more accurate.

8. AWQ: Activation-aware Weight Quantization

AWQ is a Post-Training Quantization (PTQ) method like GPTQ, but it improves output accuracy by also considering the **activation distribution** during quantization.

Feature	GPTQ	AWQ
Error Modeling	Weight-only error	Weight + Activation error
Uses Hessian	☒ Yes	☒ No (simpler & faster)
Calibration Needed	☒ Yes (50–200 samples)	☒ Yes (same)
Precision	INT4 (mainly)	INT4 (mainly)
Speed	Fast	Even faster (no second-order calc)
Accuracy	High	Very High (better than GPTQ in some LLMs)
Quant Target	Weights	Weights + Activation impact
Use Case	LLaMA, GPT	LLaMA, Mistral, ChatGLM, etc.

Intuition Behind AWQ

Instead of blindly quantizing weights, **AWQ checks how each weight affects the output, after going through activation functions** (like GELU, ReLU, etc).

It then **prunes and rescales weights** so that they cause **minimum distortion** in output — especially on **important tokens** during inference.

Why is this Better Than GPTQ in Some Cases?

GPTQ	AWQ
Focus on pre-activation error	Focus on post-activation (real output) error
Uses second-order gradients (Hessian)	No gradients – lighter & faster
Slower on large models	Faster, no Hessian
Good for precision	Better for both precision & speed (especially in inference-only scenarios)

Real-Life Analogy

GPTQ = Measures how much *raw ingredients* changed

AWQ = Measures how the *final dish tastes* after cooking (activation)

✓ Summary of Mathematical Pipeline:

Step	Operation	Math
1	Activation Capture	$X \leftarrow \text{calibration data}$
2	Token Importance	Score high-impact tokens
3	Weight Scaling	$W' = \text{Scale}(W, \text{importance})$
4	Pruning (optional)	Remove low-importance weights
5	Quantization	$\hat{W} = \text{Quantize}(W')$
6	Final Output	$\hat{Y} = f(X\hat{W})$, minimize $\ f(XW) - f(X\hat{W})\ ^2$

AWQ Mathematics – Full Detailed Breakdown

Problem Setup:

In any Transformer-based LLM:

- X = Input activation matrix (tokens $\rightarrow$ hidden states)
- W = Weight matrix of a linear layer (Q, K, V, FFN etc.)
- f = Non-linear activation (GELU, ReLU etc.)
- $\hat{W}$ = Quantized weights

The output of a layer is:

$$Y = f(XW)$$

When quantized:

$$\hat{Y} = f(X\hat{W})$$

Goal of AWQ is to minimize output error:

$$\min_{\hat{W}} \|f(XW) - f(X\hat{W})\|_F^2$$

Ye GPTQ se different hai. GPTQ minimize karta hai:

$$\|XW - X\hat{W}\|_F^2 \quad (\text{before activation})$$

But AWQ includes non-linearity, so it's more realistic and better for preserving accuracy in LLMs.

Step-by-Step Explanation:

Step 1: Calibration

- 50–200 real text prompts ko model me input diya jaata hai.
- Ye prompts se **activation values** XXX collect kiye jaate hain.

Step 2: Token Importance Estimation

Some tokens contribute **more to final output** than others.

AWQ:

- Calculates per-token activation scores.

- Identifies "**important tokens**" – jinpe quantization ka impact kam se kam hona chahiye.

Step 3: Weight Rescaling & Pruning

Weight matrix W ko **token importance** ke hisaab se adjust kiya jaata hai.

Heuristic approach:

- Weights are **rescaled** and **non-important rows/columns pruned** before quantization.
- This is done **blockwise**, just like GPTQ, typically in **128-row blocks**.

$$\hat{W} = \text{Quantize}(\text{Prune}(\text{Scale}(W, \text{token importance})))$$

Step 4: INT4 Quantization

Once rescaled and pruned:

- INT4 quantization is applied.
- Same formula as standard PTQ:

$$w_q = \text{round}\left(\frac{w}{s}\right)$$

Where:

- s = scale factor for block
- w = original (rescaled) weight
- w_q = quantized weight (in INT4 range)

Why is this Better Than GPTQ in Some Cases?

GPTQ

Focus on pre-activation error

Uses second-order gradients (Hessian)

Slower on large models

AWQ

Focus on post-activation (real output) error

No gradients – lighter & faster

Faster, no Hessian

9. GGML(Georgi Gerganov's Machine Learning)

GGML: Georgi Gerganov's Machine Learning

- **GGML** originally stood for the library created by **Georgi Gerganov**, who is the creator of `llama.cpp`.
- It is **not** "GPT-Generated Model Language" (that's a common misinterpretation online).
- GGML is a **tensor library for machine learning on CPUs**, optimized for quantized inference.

Use case:

Used to run LLaMA, Whisper, GPT models on laptops/mobile devices using CPU. Initially supported .bin model files (quantized via GGML format).

<https://arxiv.org/abs/2210.17323>

1. What is GGML?

GGML stands for **Georgi Gerganov's Machine Learning** framework.

✓ It's a C-based tensor library optimized for **inference of quantized models** — especially for **CPU and Apple Silicon (Mac)**.

✗ GGML was not designed for training or GPU inference (though hacks exist).

2. Key Highlights

Feature	Description
⚙️ Language	Written in pure C (highly portable)
🎯 Focus	Efficient inference of LLMs with quantized weights
💾 Quantization	INT8, INT4, custom quantizations like Q4_0, Q4_K, etc.
🧠 Model Types	Works well with decoder-only LLMs like LLaMA, Falcon
💻 Hardware Optimized	CPU (x86, ARM), Apple Silicon (Metal backend)

3. What GGML Is Not Meant For

✗ Not meant for	Reason
GPU acceleration (by default)	No CUDA / ROCm support in core
Model training	No backward pass / autograd
Fine-tuning	Only forward/inference supported

4. Why GGML Became Popular

- ◇ It enabled **local inference of LLaMA models** on laptops.
- ◇ Used in tools like:
 - **llama.cpp** (main client)
 - **text-generation-webui**
 - **koboldcpp**

5. GGML vs GGUF

Feature	GGML Format	GGUF Format
◇ File type	<i>.bin + external tokenizer files</i>	<i>.gguf (self-contained)</i>
◇ Metadata	<i>Minimal</i>	<i>Rich + embedded</i>
◇ Tokenizer	<i>Separate file needed</i>	<i>Embedded</i>
◇ Quant formats	<i>Limited</i>	<i>Wide support (Q2_K to Q8_0)</i>
◇ Versioning	<i>No formal versioning</i>	<i>GGUF has versioning</i>

☑ **GGUF succeeded GGML** and is now the preferred format for llama.cpp.

6. GGML Format Structure

GGML stores models in .bin files with this format:

[Header]

- └─ Magic bytes: GGML
- └─ Version info (optional)

[Model Info]

- └─ Dimensions, number of layers, vocab size

[Tensor Blobs]

- └─ Each tensor has name, shape, data type (quantized), actual values

Each weight is stored in its **quantized form**, e.g., INT4 or INT8.

7. Quantization Support in GGML

Quantization Bit Width Use Case

Q4_0	4-bit	Basic compression
Q4_K	4-bit	Smarter quant (like AWQ)
Q5_0	5-bit	Balanced speed & accuracy
Q8_0	8-bit	Near-FP16 quality
F16 / F32	16/32-bit	Full precision

Note: GGML does **not** use standard torch.quantization. It has its **own custom quant** schemes optimized for memory and cache locality.

8. Use with llama.cpp

Although GGML can be used alone, it's mostly used **via llama.cpp**, which wraps its functionality for:

- Text generation
- Prompt processing
- Streaming outputs
- GPU offloading (partial)

- Quantized inference on CPU/Mac

Example command:

`./main -m model.bin -p "Explain quantization in LLMs."`

9. Tooling

Tool	Purpose
<code>convert.py</code>	Convert Hugging Face models to GGML
<code>llama.cpp</code>	CLI wrapper to run GGML models
<code>text-generation-webui</code>	Web interface using GGML backend

10. Transition to GGUF

- GGML format is **deprecated** for new projects.
- Convert your GGML `.bin` models to `.gguf` using scripts like:
`python3 convert-llama-ggml-to-gguf.py`

GGUF adds:

- Tokenizer support
- Quant format standardization
- Model info + metadata
- Compatibility with newer LLaMA variants

Summary

GGML	GGUF
Old quantized format	New unified format
<code>.bin</code> files	<code>.gguf</code> files
No tokenizer inside	Tokenizer embedded
No versioning	Versioned and standardized
Still works	Recommended to migrate 

10. GGUF(Gerganov's General Unified Format):

GGUF (GGML Unified Format) is a standardized file format designed for storing and deploying quantized large language models (LLMs), primarily used within the `llama.cpp` ecosystem. **“GGUF (a unified binary format from the GGML/llama.cpp ecosystem) is a file format that streamlines the storage and deployment of quantized large language models (LLMs).**

GGUF: GPT-Generated Unified Format

- GGUF stands for **GPT-Generated Unified Format**, though some sources also say it loosely means **Gerganov's General Unified Format**.
- It's the **successor to GGML format**.
- Designed by the **llama.cpp community**, it's a **self-describing file format** to make quantized LLM deployment easier.

Use case:

Stores model weights + **complete metadata** (tokenizer, vocab, rope params, etc.) so tools don't need manual flags. Hugging Face and llama.cpp fully support GGUF.

1. What is GGUF?

GGUF stands for **Gerganov's General Unified Format**.

It is a new binary file format introduced as a **replacement for GGML models** for efficient storage and loading of quantized large language models (LLMs), especially in the llama.cpp ecosystem.

It standardizes how models are stored, along with all metadata, tokenizer, quantization details, etc., into **one portable file**.

2. Why GGUF?

GGUF solves key problems of previous formats like GGML:

Problem in GGML

Separate files for tokenizer and metadata

Not versioned properly

Hard to inspect

Lacked standard tokenizer info

Limited support for new quantization types

GGUF Solution

✓ Single file with everything packed

✓ Formal versioning and backward compatibility

✓ Human-readable metadata using gguf-tool

✓ Tokenizer, vocab, and pre-tokenizer embedded

✓ Native support for INT4, Q8_0, Q6_K, etc.

3. GGUF File Contents

Each .gguf file stores:

Component

Description

model parameters Hidden size, head count, vocab size, etc.

tensor weights Quantized tensors like Q/K/V, FFN

tokenizer config Pre-tokenizer, vocab, tokenizer type

metadata Format version, model source, training notes

quantization type E.g., Q4_K, Q8_0, etc.

4. Supported Models

GGUF works best with models using the **Transformers architecture**, such as:

- **LLaMA** / LLaMA 2 / LLaMA 3
- Falcon
- Mistral
- BLOOM / BLOOMZ
- GPT-J / GPT-NeoX
- Phi-2
- StarCoder

5. GGUF Format Structure (Internals)

At a low level, GGUF is a **binary format** with these parts:

[Header]

└─ Magic Number: GGUF

└─ Version: v1 or v2

[Metadata]

└─ Model name, tokenizer info, quantization method

[Tensors]

└─ List of all quantized model weights

└─ Each tensor has: name, shape, data type, actual data

[Tokenizer Info]

└─ Tokenizer type (BPE, SentencePiece, etc.)

└─ Special tokens, pre-tokenizer config, etc.

6. GGUF Quantization Types Supported

Type	Bits	Notes
------	------	-------

Q4_0	4-bit	Basic 4-bit weights
------	-------	---------------------

Q4_K	4-bit	Activation-aware 4-bit (used in AWQ)
------	-------	--------------------------------------

Q5_0	5-bit	Balanced accuracy/speed
------	-------	-------------------------

Q8_0	8-bit	Near FP16 accuracy
------	-------	--------------------

F16	16-bit	Half-precision
-----	--------	----------------

F32	32-bit	Full precision
-----	--------	----------------

7. Tooling & Ecosystem

Tool	Purpose
------	---------

llama.cpp	Main runtime for .gguf models
-----------	-------------------------------

convert.py	Script to convert Hugging Face → GGUF
------------	---------------------------------------

gguf-tool	Inspect, edit, and compare GGUF files
-----------	---------------------------------------

text-generation-webui	UI-based serving of GGUF
-----------------------	--------------------------

KoboldCpp, llamafile	Other backends that support GGUF
----------------------	----------------------------------

8. Real-World Workflow

Step 1: Quantize using AutoGPTQ or AWQ

Step 2: Export to GGUF using Transformers + llama.cpp scripts

Step 3: Run on CPU/Mac/low-VRAM GPU using llama.cpp

llama.cpp --model model.gguf --prompt "Explain black holes."

9. Pros & Limitations

✓ Pros:

- Compact single file
- Easy deployment across devices
- Standardized for llama.cpp ecosystem
- Compatible with many quant formats

⚠ Limitations:

- Mostly optimized for decoder-only (causal) transformers
- Not natively supported by Hugging Face libraries (only conversion possible)

10. Future-Proofing

GGUF is under **active development**. It's expected to:

- Add support for **speculative decoding**
- Allow **multi-model packing** (e.g., Mixture of Experts)
- Embed custom metadata for retrieval, agents, etc.

11. Practical Example with LLMs

<https://chatgpt.com/share/689f90fa-d804-8006-a1fd-a8d8c41d3a2b>